



ΠΑΝΕΠΙΣΤΗΜΙΟ ΘΕΣΣΑΛΙΑΣ

Σχολή Τεχνολογίας

Τμήμα Ψηφιακών Συστημάτων

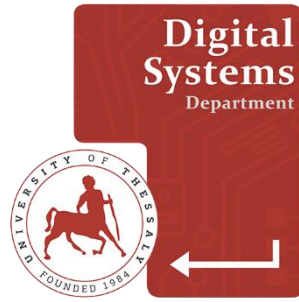
Ανάπτυξη Εκπαιδευτικού Παιχνιδιού για την Εκμάθηση Python μέσω Διαδραστικών Puzzles

ΠΤΥΧΙΑΚΗ ΕΡΓΑΣΙΑ

Βασίλειος Λαδόπουλος (ΑΜ: [removed])

Επιβλέπων: Πέτρος Λάμπας, Καθηγητής

ΛΑΡΙΣΑ, Ιούνιος 2026



UNIVERSITY OF THESSALY

School of Technology

Digital Systems Department

Development of an Educational Game for Learning Python through Interactive Puzzles

BACHELOR THESIS

Vasileios Ladopoulos (RN: [removed])

Supervisor: *Petros Lampsas, Professor*

LARISSA, June 2026

Υπεύθυνη Δήλωση περί Ακαδημαϊκής Δεοντολογίας και Πνευματικών Δικαιωμάτων

Με πλήρη επίγνωση των συνεπειών του νόμου περί πνευματικών δικαιωμάτων, δηλώνω ρητά ότι η παρούσα πτυχιακή εργασία, καθώς και τα ηλεκτρονικά αρχεία και πηγαίοι κώδικες που αναπτύχθηκαν ή τροποποιήθηκαν στα πλαίσια αυτής της εργασίας, αποτελεί αποκλειστικά προϊόν προσωπικής μου εργασίας, δεν προσβάλλει κάθε μορφής δικαιώματα διανοητικής ιδιοκτησίας, προσωπικότητας και προσωπικών δεδομένων τρίτων, δεν περιέχει έργα/εισφορές τρίτων για τα οποία απαιτείται άδεια των δημιουργών/δικαιούχων και δεν είναι προϊόν μερικής ή ολικής αντιγραφής, οι πηγές δε που χρησιμοποιήθηκαν περιορίζονται στις βιβλιογραφικές αναφορές και μόνον, και πληρούν τους κανόνες της επιστημονικής παράθεσης. Τα σημεία όπου έχω χρησιμοποιήσει ιδέες, κείμενο, αρχεία ή/και πηγές άλλων συγγραφέων, αναφέρονται ευδιάκριτα στο κείμενο με την κατάλληλη παραπομπή και η σχετική αναφορά περιλαμβάνεται στο τμήμα των βιβλιογραφικών αναφορών με πλήρη περιγραφή.

Αναλαμβάνω πλήρως, ατομικά και προσωπικά, όλες τις νομικές και διοικητικές συνέπειες που δύναται να προκύψουν στην περίπτωση κατά την οποία αποδειχθεί, διαχρονικά, ότι η εργασία αυτή ή τμήμα της δεν μου ανήκει διότι είναι προϊόν λογοκλοπής.

Ο/Η Δηλών/ούσα



Βασίλειος Λαδόπουλος

22/6/2026

Εγκρίνεται από την Επιτροπή Εξέτασης:

Επιβλέπων	Πέτρος Λάμπας Καθηγητής, Τμήμα Ψηφιακών Συστημάτων, Πανεπιστήμιο Θεσσαλίας
Μέλος	Ονοματεπώνυμο Μέλους 1 Βαθμίδα/ιδιότητα μέλους 1, Τμήμα/Ιδρυμα μέλους 1
Μέλος	Ονοματεπώνυμο Μέλους 2 Βαθμίδα/ιδιότητα μέλους 2, Τμήμα/Ιδρυμα μέλους 2

Ημερομηνία έγκρισης: 21-1-2026

Περίληψη

Η παρούσα πτυχιακή εργασία επικεντρώνεται στον σχεδιασμό και την ανάπτυξη ενός ψηφιακού παιχνιδιού μικρής κλίμακας για την εκμάθηση βασικών εννοιών προγραμματισμού στη γλώσσα Python, με στόχο μαθητές ηλικίας 12-16 ετών και αρχάριους χρήστες χωρίς προηγούμενη εμπειρία. Η ανάγκη για τη δημιουργία ενός τέτοιου περιβάλλοντος προέκυψε από τη δυσκολία που συχνά αντιμετωπίζουν οι αρχάριοι κατά την πρώτη τους επαφή με τις βασικές δομές προγραμματισμού, καθώς και από την παιδαγωγική αξία που έχουν οι διαδραστικές και οι μορφές παιχνιδιού μάθησης στην ενίσχυση του ενδιαφέροντος και της συμμετοχής των μαθητών.

Η εργασία βασίστηκε στη λογική της μάθησης μέσω επίλυσης προβλημάτων, σύμφωνα με την οποία ο εκπαιδευόμενος δεν παραμένει παθητικός δέκτης γνώσης, αλλά καλείται να αλληλεπιδράσει ενεργά με διαδοχικές προκλήσεις και να οικοδομήσει τη γνώση του μέσα από τη συγγραφή πραγματικού κώδικα. Για τον σκοπό αυτό αναπτύχθηκε ένα desktop παιχνίδι με χρήση της βιβλιοθήκης Pygame, το οποίο περιλαμβάνει οκτώ επίπεδα προοδευτικής δυσκολίας. Κάθε επίπεδο εστιάζει σε μία βασική προγραμματιστική έννοια, όπως η σειριακή εκτέλεση εντολών, οι μεταβλητές, οι αριθμητικές πράξεις, οι συνθήκες και οι επαναλήψεις, ενώ το τελικό επίπεδο απαιτεί τον συνδυασμό όλων των προηγούμενων γνώσεων.

Κατά την ανάπτυξη της εφαρμογής δόθηκε έμφαση στη σαφήνεια του γραφικού περιβάλλοντος, στην άμεση ανατροφοδότηση και στη δημιουργία ασφαλούς περιβάλλοντος εκτέλεσης κώδικα, ώστε ο χρήστης να μπορεί να πειραματίζεται χωρίς κίνδυνο. Η εργασία ολοκληρώθηκε με την ανάπτυξη ενός λειτουργικού εκπαιδευτικού παιχνιδιού, το οποίο αξιοποιεί διαδραστικά puzzles για την εισαγωγή βασικών εννοιών προγραμματισμού στην Python. Με αυτόν τον τρόπο, διαπιστώθηκε ότι η συνδυαστική χρήση κώδικα, σταδιακής καθοδήγησης και άμεσης ανατροφοδότησης μπορεί να βοηθήσει ουσιαστικά στην κατανόηση των βασικών δομών της γλώσσας και να ενισχύσει την αυτοπεποίθηση των αρχάριων χρηστών.

Abstract

This thesis focuses on the design and development of a small-scale digital game for learning fundamental programming concepts in Python, targeting students aged 12-16 and beginners with no prior experience. The need to create such an environment arose from difficulties beginners often face during their initial contact with basic programming structures, as well as the pedagogical value of interactive and game-based learning models in enhancing students' interest and engagement.

The project is based on the principles of problem-solving learning, where the learner does not remain a passive recipient of knowledge but is instead prompted to actively interact with sequential challenges and build knowledge by writing actual code. For this purpose, a desktop game was developed using the Pygame library, featuring eight levels of progressive difficulty. Each level focuses on a core programming concept, such as sequential execution, variables, arithmetic operations, conditional statements, and loops, while the final level requires a combination of all previously acquired knowledge.

During the development of the application, emphasis was placed on the clarity of the graphic user interface, immediate feedback, and the creation of a secure code execution environment, allowing users to experiment safely. The thesis was completed in a fully functional educational game that utilizes interactive puzzles to introduce core Python programming concepts. Ultimately, it was demonstrated that the combined use of code, step-by-step guidance, and immediate feedback can significantly help in understanding basic language structures and boost the confidence of beginners users.

Ευχαριστίες

Θα ήθελα να ευχαριστήσω θερμά τον επιβλέποντα καθηγητή μου για την καθοδήγηση, τη βοήθεια και τις χρήσιμες συμβουλές του κατά τη διάρκεια της εκπόνησης της παρούσας πτυχιακής εργασίας. Η υποστήριξή του υπήρξε πολύτιμη σε όλα τα στάδια της διαδικασίας.

Επίσης, ευχαριστώ την οικογένειά μου για τη στήριξη, την κατανόηση και την υπομονή της σε όλη τη διάρκεια των σπουδών μου. Η συμβολή της ήταν σημαντική και ουσιαστική.

Βασίλειος Λαδόπουλος

22/6/202

Περιεχόμενα

Ξ

ΠΕΡΙΛΗΨΗ	VII
ABSTRACT	IX
ΕΥΧΑΡΙΣΤΙΕΣ.....	XI
ΠΕΡΙΕΧΟΜΕΝΑ.....	XIII
1 ΕΙΣΑΓΩΓΗ	1
1.1 ΑΝΤΙΚΕΙΜΕΝΟ ΤΗΣ ΠΤΥΧΙΑΚΗΣ	1
1.2 ΚΙΝΗΤΡΟ ΚΑΙ ΑΝΑΓΚΗ	2
1.3 ΔΟΜΗ ΤΗΣ ΕΡΓΑΣΙΑΣ	3
2 ΠΑΙΧΝΙΔΟΚΕΝΤΡΙΚΗ ΜΑΘΗΣΗ.....	5
2.1 ΟΡΙΣΜΟΣ ΚΑΙ ΘΕΩΡΗΤΙΚΟ ΠΛΑΙΣΙΟ	5
2.2 ΠΛΕΟΝΕΚΤΗΜΑΤΑ ΤΗΣ ΠΑΙΧΝΙΔΟΚΕΝΤΡΙΚΗΣ ΜΑΘΗΣΗΣ.....	7
2.3 ΠΡΟΚΛΗΣΕΙΣ ΣΤΗΝ ΥΛΟΠΟΙΗΣΗ ΤΗΣ ΠΑΙΧΝΙΔΟΚΕΝΤΡΙΚΗΣ ΜΑΘΗΣΗΣ.....	8
3 ΤΟ ΠΑΙΧΝΙΔΙ “REDDY’S QUEST”	11
3.1 ΗΛΙΚΙΑΚΗ ΟΜΑΔΑ ΚΑΙ ΠΛΑΙΣΙΟ	11
3.2 ΣΧΕΔΙΑΣΤΙΚΟ ΣΚΕΠΤΙΚΟ.....	12
3.3 ΤΕΧΝΟΛΟΓΙΕΣ ΥΛΟΠΟΙΗΣΗΣ ΚΑΙ ΤΕΚΜΗΡΙΩΣΗ ΕΠΙΛΟΓΩΝ	13
3.4 ΡΟΗ ΚΑΤΑΣΤΑΣΕΩΝ ΚΑΙ ΔΙΕΠΑΦΗ	14
3.5 ΑΦΗΓΗΜΑΤΙΚΟ ΠΛΑΙΣΙΟ ΚΑΙ ΔΟΜΗ ΕΠΙΠΕΔΩΝ	15
3.6 ΑΡΧΙΤΕΚΤΟΝΙΚΗ ΤΟΥ ΚΩΔΙΚΑ.....	16
3.7 ΜΗΧΑΝΙΣΜΟΣ ΑΣΦΑΛΟΥΣ ΕΚΤΕΛΕΣΗΣ ΚΩΔΙΚΑ (SANDBOX)	21
3.8 ΜΗΧΑΝΙΣΜΟΣ ΑΞΙΟΛΟΓΗΣΗΣ ΤΟΥ ΚΩΔΙΚΑ ΧΡΗΣΤΗ.....	23
3.9 ΣΥΣΤΗΜΑ ΒΑΘΜΟΛΟΓΗΣΗΣ ΚΑΙ ΒΟΗΘΗΜΑΤΩΝ.....	24
3.10 ΖΗΤΗΜΑΤΑ ΠΟΥ ΑΝΤΙΜΕΤΩΠΙΣΤΗΚΑΝ ΚΑΤΑ ΤΗΝ ΥΛΟΠΟΙΗΣΗ.....	25
3.11 ΕΝΔΕΙΚΤΙΚΑ ΣΤΙΓΜΙΟΤΥΠΑ	27

3.12 ΑΠΑΙΤΗΣΕΙΣ ΚΑΙ ΕΚΤΕΛΕΣΗ	31
4 ΜΕΛΛΟΝΤΙΚΗ ΕΡΓΑΣΙΑ	33
4.1 ΑΞΙΟΛΟΓΗΣΗ ΣΕ ΠΡΑΓΜΑΤΙΚΕΣ ΣΥΝΘΗΚΕΣ ΧΡΗΣΗΣ ΚΑΙ ΚΑΤΑΓΡΑΦΗ ΤΗΛΕΜΕΤΡΙΑΣ.....	33
4.2 ΕΠΕΚΤΑΣΗ ΤΟΥ ΠΕΡΙΕΧΟΜΕΝΟΥ ΚΑΙ ΠΡΟΣΑΡΜΟΣΤΙΚΗ ΔΥΣΚΟΛΙΑ	34
4.3 ΒΕΛΤΙΩΣΕΙΣ ΔΙΕΠΑΦΗΣ, ΠΡΟΣΒΑΣΙΜΟΤΗΤΑΣ ΚΑΙ ΕΡΓΑΛΕΙΑ ΥΠΟΣΤΗΡΙΞΗΣ ΕΚΠΑΙΔΕΥΤΙΚΟΥ.....	35
4.4 ΑΣΦΑΛΕΙΑ, ΜΗΧΑΝΙΣΜΟ ΑΞΙΟΛΟΓΗΣΗΣ ΚΑΙ ΤΕΧΝΙΚΗ ΕΠΕΚΤΑΣΗ	36
ΒΙΒΛΙΟΓΡΑΦΙΑ.....	39
ΠΑΡΑΡΤΗΜΑ Α	41
A.1 ΚΩΔΙΚΑΣ ΠΛΗΡΗΣ ΛΙΣΤΑΣ LEVELS	41
A.2 ΜΗΧΑΝΙΣΜΟΣ ΑΣΦΑΛΟΥΣ ΕΚΤΕΛΕΣΗΣ ΚΩΔΙΚΑΣ (SANDBOX).....	43

1 Εισαγωγή

Το συγκεκριμένο κεφάλαιο εισάγει τον αναγνώστη στο αντικείμενο της πτυχιακής εργασίας, εξηγώντας το πρόβλημα που κλήθηκε να αντιμετωπιστεί, το σκεπτικό πίσω από την επιλογή του συγκεκριμένου τρόπου προσέγγισης, καθώς και τις τεχνολογίες που αξιοποιήθηκαν για την υλοποίησή του. Στο τέλος του κεφαλαίου παρουσιάζεται συνοπτικά η δομή που ακολουθεί η υπόλοιπη εργασία.

1.1 Αντικείμενο της πτυχιακής

Η εκμάθηση προγραμματισμού, εδώ και χρόνια, είναι ένα από τα πιο απαιτητικά αντικείμενα στην εκπαίδευση, τόσο σε επίπεδο Γυμνασίου-Λυκείου όσο και σε επίπεδο εισαγωγικών μαθημάτων στην τριτοβάθμια εκπαίδευση. Η αφηρημένη φύση εννοιών όπως οι μεταβλητές, οι συνθήκες και οι επαναλήψεις, σε συνδυασμό με τη σύνταξη των γλωσσών προγραμματισμού, δημιουργεί συχνά ένα αίσθημα αποθάρρυνσης στους αρχάριους, ιδίως σε μαθητές χωρίς προηγούμενη επαφή με τον προγραμματισμό. Το φαινόμενο αυτό είναι ιδιαίτερα έντονο στις πρώτες ώρες επαφής με μια γλώσσα προγραμματισμού, όπου ένα απλό συντακτικό λάθος (π.χ. λάθος εσοχή σε Python) μπορεί να λειτουργήσει αρνητικά για έναν μαθητή που δεν έχει ακόμα αναπτύξει «νοητικό μοντέλο» του πώς εκτελείται ένα πρόγραμμα (Lister et al., 2004).

Παράλληλα, η ίδια η φύση του προγραμματισμού, δηλαδή η ανάγκη να «σκέφτεται» κανείς αλγοριθμικά, να αναλύει ένα πρόβλημα σε διακριτά βήματα και να προβλέπει τη συμπεριφορά ενός προγράμματος πριν καν το εκτελέσει, δεν διδάσκεται εύκολα μέσα από παθητική παρακολούθηση θεωρίας. Χρειάζεται ενεργή εμπλοκή, δοκιμή και άμεση ανατροφοδότηση, στοιχεία που παραδοσιακά εντοπίζονται και στο σχεδιασμό των παιχνιδιών (Gros, 2007).

Αντικείμενο αυτής της πτυχιακής εργασίας είναι ο σχεδιασμός, η ανάπτυξη και η τεκμηρίωση ενός εκπαιδευτικού παιχνιδιού μικρής κλίμακας, με τίτλο "Reddy's Quest", το οποίο στοχεύει στην εισαγωγή βασικών εννοιών προγραμματισμού στη γλώσσα Python μέσα από ένα διαδραστικό, παιχνιδιοκεντρικό περιβάλλον. Το παιχνίδι απευθύνεται σε μαθητές Γυμνασίου–Λυκείου (12–16 ετών), καθώς και σε κάθε αρχάριο χρήστη

χωρίς προηγούμενη εμπειρία στον προγραμματισμό. Η επιλογή αυτής της ηλικιακής ομάδας δεν είναι τυχαία: πρόκειται για την περίοδο κατά την οποία εισάγονται συνήθως τα πρώτα μαθήματα πληροφορικής/προγραμματισμού στο ελληνικό εκπαιδευτικό σύστημα, γεγονός που καθιστά το εργαλείο δυνητικά αξιοποιήσιμο ως συμπληρωματικό υλικό στη σχολική τάξη ή για αυτόνομη μελέτη.

Η παιδαγωγική προσέγγιση που χρησιμοποιήθηκε βασίζεται στη μάθηση μέσω επίλυσης προβλημάτων (problem-based learning): αντί ο χρήστης να παρακολουθεί παθητικά θεωρία, καλείται να γράψει πραγματικό, εκτελέσιμο κώδικα Python προκειμένου να λύσει διαδραστικά puzzles αυξανόμενης δυσκολίας, λαμβάνοντας άμεση ανατροφοδότηση για την ορθότητα της λύσης του. Κάθε puzzle είναι σχεδιασμένο έτσι ώστε να απαιτεί τη χρήση συγκεκριμένης προγραμματιστικής δομής (π.χ. δεν αρκεί να τυπωθεί το σωστό αποτέλεσμα, πρέπει να παραχθεί μέσω της δομής που διδάσκει το συγκεκριμένο επίπεδο), ώστε η επιτυχία στο puzzle να αντιστοιχεί πραγματικά στην κατάκτηση της έννοιας και όχι σε τυχαία ή «πονηρή» λύση (Papert, 1980; Wiggins & McTighe, 2005).

Η κύρια συνεισφορά της παρούσας εργασίας είναι η ανάπτυξη ενός αυτόνομου εκπαιδευτικού παιχνιδιού για την εκμάθηση βασικών εννοιών της Python, το οποίο συνδυάζει πραγματικό προγραμματισμό με στοιχεία παιχνιδοποίησης και αυτόματη αξιολόγηση της λύσης του χρήστη μέσω συντακτικής ανάλυσης του κώδικα. Το παιχνίδι λειτουργεί ως εφαρμογή επιφάνειας εργασίας και μπορεί να αξιοποιηθεί τόσο για αυτόνομη μελέτη όσο και ως υποστηρικτικό εκπαιδευτικό εργαλείο.

1.2 Κίνητρο και ανάγκη

Η επιλογή του συγκεκριμένου θέματος προέκυψε από δύο κύριες παρατηρήσεις. Πρώτον, οι παραδοσιακές μέθοδοι διδασκαλίας προγραμματισμού (διαφάνειες, στατικά παραδείγματα, ασκήσεις σε χαρτί) δυσκολεύονται να διατηρήσουν το ενδιαφέρον νεαρών μαθητών, οι οποίοι είναι εξοικειωμένοι με διαδραστικά, οπτικά περιβάλλοντα (βιντεοπαιχνίδια, εφαρμογές, πλατφόρμες με άμεση ανατροφοδότηση). Η παθητική έκθεση σε θεωρία, χωρίς άμεση δοκιμή και επιβεβαίωση, τείνει να παράγει επιφανειακή κατανόηση.

Δεύτερον, από μια σύντομη επισκόπηση υπαρχόντων εργαλείων εκμάθησης προγραμματισμού μέσω παιχνιδιού, διαπιστώθηκε ένα κενό ανάμεσα σε δύο «άκρα»: από την μία έχουμε εργαλεία όπως το Scratch, που απευθύνονται σε πολύ μικρότερες ηλικίες και χρησιμοποιούν οπτικό προγραμματισμό αντί για πραγματικό κώδικα κειμένου, με

αποτέλεσμα να μην προετοιμάζουν επαρκώς τον μαθητή για τη μετάβαση σε μια πραγματική γλώσσα κειμένου. Από την άλλη, έχουμε πλατφόρμες άσκησης κώδικα (π.χ. online judge) που, αν και χρησιμοποιούν πραγματικό κώδικα, στερούνται στα επίπεδα της αφήγησης ή των στοιχείων παιχνιδιού που κρατούν σε ενδιαφέρον τον αρχάριο μαθητή (Resnick et al., 2009).

Το “Reddy’s Quest” προσπαθεί να καλύψει ακριβώς αυτό το ενδιάμεσο κενό: συνδυάζοντας πραγματική σύνταξη Python με ένα αφηγηματικό πλαίσιο (τον χαρακτήρα Reddy που προσπαθεί να φτάσει στον «Πύργο της Python» για να βρει τον «Κρύσταλλο της Γνώσης») και με μηχανισμούς παιχνιδιού (συστήματα αστεριών ανά επίπεδο, προαιρετικά βοηθήματα, ηχητική και οπτική ανατροφοδότηση) που ενισχύουν το κίνητρο του παίκτη να ολοκληρώσει κάθε επίπεδο και να προχωρήσει στο επόμενο. Η πρόοδος του χρήστη (αστέρια ανά πίστα, συνολικά αστέρια) παραμένει ορατή σε όλη τη διάρκεια του παιχνιδιού μέσω ενός χάρτη/γραμμής προόδου, ώστε να ενισχύεται η αίσθηση της εξέλιξης.

1.3 Δομή της εργασίας

Η υπόλοιπη της εργασία οργανώνεται ως εξής: Το Κεφάλαιο 2 παρουσιάζει το θεωρητικό πλαίσιο της Παιχνιδοκεντρικής Μάθησης (Games-based Learning), τα πλεονεκτήματα και τις προκλήσεις που περιλαμβάνει η υλοποίησή της. Το Κεφάλαιο 3 περιγράφει αναλυτικά το παιχνίδι που αναπτύχθηκε: τον σχεδιασμό των 8 επιπέδων, την αρχιτεκτονική του κώδικα, τους μηχανισμούς ασφαλούς εκτέλεσης (sandboxing) και αξιολόγησης του κώδικα του χρήστη, καθώς και ενδεικτικά στιγμιότυπα (screenshots). Το Κεφάλαιο 4 συζητά κατευθύνσεις μελλοντικής εργασίας, με έμφαση στην αξιολόγηση του παιχνιδιού σε πραγματικές συνθήκες χρήσης. Τέλος, το Κεφάλαιο 5 παραθέτει τη βιβλιογραφία που χρησιμοποιήθηκε.

2 Παιχνιδοκεντρική Μάθηση

Στην συνέχεια παρουσιάζεται το θεωρητικό πλαίσιο στο οποίο εντάσσεται η ανάπτυξη του παιχνιδιού, "Reddy's Quest". Αρχικά, ορίζεται η έννοια της Παιχνιδοκεντρικής Μάθησης και το θεωρητικό της υπόβαθρο, με αναφορά σε παραδείγματα εφαρμογής της στην εκμάθηση προγραμματισμού. Συζητούνται τα πλεονεκτήματα που προσφέρει η μέθοδος αυτή, όπως και οι προκλήσεις που προκύπτουν κατά την επιλογή της.

2.1 Ορισμός και θεωρητικό πλαίσιο

Η Παιχνιδοκεντρική Μάθηση (Games-based Learning, εφεξής ΠΜ) αναφέρεται στη χρήση παιχνιδιών, είτε σχεδιασμένων αποκλειστικά για εκπαιδευτικό σκοπό είτε προσαρμοσμένων από ήδη υπάρχοντα ψυχαγωγικά παιχνίδια, ως μέσου διδασκαλίας γνώσεων, δεξιοτήτων ή στάσεων (Prensky, 2001). Η προσέγγιση αυτή δεν ταυτίζεται με τη «γενική χρήση παιχνιδιών στην τάξη», αλλά αφορά συστήματα στα οποία οι μηχανισμοί του ίδιου του παιχνιδιού (κανόνες, στόχοι, ανταμοιβές, αφήγηση) είναι σχεδιασμένοι ώστε να ενσωματώνουν και να ενισχύουν συγκεκριμένους μαθησιακούς στόχους, και όχι απλώς να φαίνονται περισσότερο ελκυστικά σε σχέση με ένα παραδοσιακό εκπαιδευτικό υλικό.

Είναι χρήσιμο, στο σημείο αυτό, να γίνει μια σύντομη εννοιολογική διάκριση ανάμεσα στην ΠΜ και τη λεγόμενη «παιχνιδοποίηση» (gamification). Η παιχνιδοποίηση αφορά την ενσωμάτωση επιμέρους στοιχείων παιχνιδιού (π.χ. πόντοι, βραβεία) σε ένα μη-παιγνιώδες πλαίσιο, με στόχο κυρίως την ενίσχυση του κινήτρου συμμετοχής (Deterding et al., 2011). Αντίθετα, στην ΠΜ ολόκληρη η μαθησιακή εμπειρία εκτυλίσσεται μέσα σε ένα πλήρες παιχνίδι, με δικό του αφηγηματικό πλαίσιο, μηχανικές και σύστημα προόδου, όπως συμβαίνει και στο "Reddy's Quest": ο χρήστης δεν καλείται απλώς να συγκεντρώσει πόντους γράφοντας κώδικα, αλλά να προχωρήσει μέσα σε μια ιστορία (την πορεία του χαρακτήρα Reddy προς τον «Πύργο της Python»), όπου κάθε επίπεδο αντιστοιχεί σε μια αφηγηματική πρόκληση που λύνεται αποκλειστικά μέσω προγραμματισμού. Στη διεθνή βιβλιογραφία, παιχνίδια όπως το "Reddy's Quest", που αναπτύσσονται με σαφή, εξ

αρχής καθορισμένο εκπαιδευτικό σκοπό, εντάσσονται συχνά στην ευρύτερη κατηγορία των «σοβαρών παιχνιδιών» (serious games) (Michael & Chen, 2006).

Το θεωρητικό υπόβαθρο της ΠΜ αντλεί κυρίως από δύο ρεύματα της παιδαγωγικής επιστήμης. Το πρώτο είναι ο εποικοδομισμός (constructivism) και ειδικότερα ο κονστρουκτιβισμός (constructionism) του Papert (1980), σύμφωνα με τον οποίο η μάθηση είναι αποτελεσματικότερη όταν ο μαθητευόμενος κατασκευάζει ενεργά νοητικά μοντέλα μέσα από την πράξη και όχι μέσα από την παθητική μετάδοση γνώσης. Το δεύτερο είναι η θεωρία της ροής (Flow Theory) του Csikszentmihalyi (1990), η οποία περιγράφει την ψυχολογική κατάσταση πλήρους εμπύθισης και συγκέντρωσης που βιώνει ένα άτομο όταν η δυσκολία μιας δραστηριότητας βρίσκεται σε ισορροπία με το επίπεδο δεξιοτήτων του, μια κατάσταση που τα καλά σχεδιασμένα παιχνίδια επιδιώκουν συστηματικά να προκαλέσουν μέσω της σταδιακής αύξησης της δυσκολίας (Kiili, 2005). Στο ίδιο πλαίσιο, ο Gee (2003) περιέγραψε ένα σύνολο «αρχών μάθησης» που εντοπίζονται στα βιντεοπαιχνίδια (π.χ. άμεση ανατροφοδότηση, ασφαλής εξερεύνηση μέσω δοκιμής-λάθους, διαβαθμισμένη δυσκολία), υποστηρίζοντας ότι οι αρχές αυτές μπορούν να μεταφερθούν αποτελεσματικά και σε τυπικά εκπαιδευτικά περιβάλλοντα.

Ένα ακόμη χρήσιμο θεωρητικό εργαλείο για την κατανόηση του πώς ένα παιχνίδι μπορεί και μετατρέπει τη διασκέδαση σε μάθηση είναι το μοντέλο εισόδου - διεργασίας - αποτελέσματος (Input - Process - Outcome model) των Garris, Ahlers & Driskell (2002). Σύμφωνα με το μοντέλο αυτό, τα χαρακτηριστικά σχεδιασμού ενός παιχνιδιού (π.χ. φαντασία, κανόνες, στόχοι, πρόκληση) ενεργοποιούν έναν επαναληπτικό κύκλο κρίσης - συμπεριφοράς - ανατροφοδότησης (judgment - behavior - feedback loop) στον παίκτη, ο οποίος με τη σειρά του οδηγεί σε μαθησιακά αποτελέσματα. Στο "Reddy's Quest", ο ρόλο αυτού του «κύκλου ανατροφοδότησης» τον αναλαμβάνει ο συνδυασμός της άμεσης οπτικοακουστικής απόκρισης (επιτυχία/αποτυχία επιπέδου).

Τέλος, αξίζει να σημειωθεί ότι η ΠΜ δεν αποτελεί ενιαία, μονολιθική μέθοδο, αλλά ένα φάσμα προσεγγίσεων που διαφοροποιούνται ως προς το βαθμό «αφομοίωσης» του μαθησιακού περιεχομένου μέσα στη μηχανική του παιχνιδιού. Σε ένα άκρο βρίσκονται παιχνίδια όπου η γνώση παρουσιάζεται ξεχωριστά από την παιγνιώδη δράση (π.χ. quiz ανάμεσα σε επίπεδα δράσης), ενώ στο άλλο άκρο βρίσκονται παιχνίδια στα οποία η ίδια η ενέργεια που προωθεί την πλοκή ταυτίζεται πλήρως με τη μαθησιακή πράξη, μια προσέγγιση που η βιβλιογραφία θεωρεί γενικά πιο αποτελεσματική, καθώς αποφεύγει τον διαχωρισμό ανάμεσα σε παιχνίδι και μάθημα (Gee, 2003).

2.2 Πλεονεκτήματα της Παιχνιδοκεντρικής Μάθησης

Η ΠΜ προσφέρει αρκετά σημαντικά οφέλη, τα οποία γίνονται ιδιαίτερα εμφανή όταν εφαρμόζεται στην εκμάθηση προγραμματισμού:

- Ενίσχυση εσωτερικού κινήτρου (intrinsic motivation): Σύμφωνα με τον Malone (1980), τα παιχνίδια που ενσωματώνουν πρόκληση, φαντασία και περιέργεια ενισχύουν κίνητρο συμμετοχής του μαθητευόμενου, με αποτέλεσμα μεγαλύτερη επιμονή απέναντι στη δυσκολία, ακόμη και όταν δεν υπάρχει εξωτερική «υποχρέωση» (π.χ. βαθμός). Στο "Reddy's Quest", η αφηγηματική πλαισίωση και το σύστημα αστεριών ανά επίπεδο αποσκοπούν ακριβώς σε αυτή τη λειτουργία, δίνοντας στον χρήστη έναν λόγο πέρα από την «τυπική» εκμάθηση για να ολοκληρώσει κάθε πίστα.
- Άμεση ανατροφοδότηση: Σε αντίθεση με πολλές παραδοσιακές μεθόδους, όπου η ανατροφοδότηση καθυστερεί (π.χ. διόρθωση εργασιών), τα εκπαιδευτικά παιχνίδια προσφέρουν στιγμιαία απόκριση στις ενέργειες του χρήστη, κάτι ιδιαίτερα σημαντικό στον προγραμματισμό, όπου η κατανόηση της σχέσης αίτιου - αποτελέσματος ανάμεσα σε ένα κομμάτι κώδικα και τη συμπεριφορά του προγράμματος είναι κρίσιμη (Gee, 2003; Lister et al., 2004). Η αμεσότητα αυτή επιτρέπει στον μαθητευόμενο να διορθώνει εσφαλμένες νοητικές αναπαραστάσεις πριν αυτές σταθεροποιηθούν, κάτι που είναι πολύ δυσκολότερο όταν η ανατροφοδότηση έρχεται ημέρες μετά τη συγγραφή του κώδικα.
- Ασφαλές περιβάλλον δοκιμής και λάθους: Το παιχνίδι επιτρέπει στον μαθητευόμενο να πειραματιστεί χωρίς τον φόβο της «αποτυχίας» με την παραδοσιακή έννοια (π.χ. βαθμολογία), αφού το λάθος αντιμετωπίζεται ως μέρος της διαδικασίας επίλυσης του puzzle. Αυτό μειώνει το άγχος επίδοσης που συχνά συνδέεται με τα πρώτα μαθήματα προγραμματισμού, ιδίως σε αρχάριους χωρίς προηγούμενη εμπειρία.
- Διαβαθμισμένη δυσκολία και εξατομικευμένη πρόοδος: Ο σχεδιασμός επιπέδων αυξανόμενης δυσκολίας επιτρέπει στον χρήστη να χτίζει σταδιακά γνώσεις πάνω σε προηγούμενες, μια αρχή που συνδέεται με τη «Ζώνη Επικείμενης Ανάπτυξης» (Zone of Proximal Development) του Vygotsky, σύμφωνα με την οποία η μάθηση είναι πιο αποτελεσματική όταν η πρόκληση βρίσκεται λίγο πάνω από το τρέχον

επίπεδο ικανότητας του μαθητευόμενου. Η δομή αυτή επιτρέπει επίσης στον χρήστη να προχωρά με τον δικό του ρυθμό, χωρίς την πίεση να «συμβαδίζει» με μια ολόκληρη τάξη.

- Ενίσχυση της αφοσίωσης και μείωση της «εγκατάλειψης»: Έρευνες δείχνουν ότι τα στοιχεία παιχνιδιού (πρόοδος, ανταμοιβή, αφήγηση) συμβάλλουν στη διατήρηση της προσοχής και του ενδιαφέροντος για μεγαλύτερο χρονικό διάστημα σε σχέση με στατικά διδακτικά υλικά, μειώνοντας το φαινόμενο της απεμπλοκής (disengagement) που παρατηρείται συχνά στα πρώτα μαθήματα προγραμματισμού (Shernoff et al., 2003; Hamari et al., 2014). Στο "Reddy's Quest", η ορατή απεικόνιση της προόδου μέσω του χάρτη/γραμμής προόδου λειτουργεί ως συνεχής υπενθύμιση της διαδρομής που έχει ήδη διανυθεί, ενισχύοντας την αίσθηση της επιτυχίας.
- Εξάσκηση υπολογιστικής σκέψης (computational thinking): Η ανάγκη ανάλυσης ενός προβλήματος σε βήματα και η αλγοριθμική επίλυση, βασικά στοιχεία της υπολογιστικής σκέψης, καλλιεργούνται φυσικά μέσα από τη διαδικασία επίλυσης puzzle, χωρίς να απαιτείται ρητή θεωρητική διδασκαλία της έννοιας (Resnick et al., 2009).

2.3 Προκλήσεις στην υλοποίηση της Παιχνιδοκεντρικής Μάθησης

Παρότι η ΠΜ παρουσιάζει σημαντικές δυνατότητες, η επιτυχής εφαρμογή της στην εκπαιδευτική πράξη συνοδεύεται από ορισμένες ουσιαστικές προκλήσεις, ειδικά όταν το αντικείμενο διδασκαλίας είναι ο προγραμματισμός. Παρακάτω παρουσιάζονται οι κυριότερες από αυτές τις προκλήσεις, οι οποίες αφορούν τόσο τον παιδαγωγικό σχεδιασμό όσο και την τεχνική υλοποίηση ενός τέτοιου περιβάλλοντος:

- Ισορροπία ανάμεσα σε παιχνίδι και μάθηση (educational - entertainment balance): Ένα από τα πιο συχνά αναφερόμενα προβλήματα στη βιβλιογραφία είναι ο κίνδυνος το παιχνίδι να γίνει είτε υπερβολικά «ψυχαγωγικό», με αποτέλεσμα ο μαθησιακός στόχος να υποβαθμιστεί σε δευτερεύον στοιχείο, είτε υπερβολικά «διδακτικό», χάνοντας τα χαρακτηριστικά που το καθιστούν ελκυστικό ως παιχνίδι (Gros, 2007). Η πρόκληση αυτή αντιμετωπίστηκε στο "Reddy's Quest" μέσω του

σχεδιασμού των puzzles ώστε η επιτυχία να προϋποθέτει ρητά τη χρήση της συγκεκριμένης προγραμματιστικής δομής κάθε επιπέδου, ώστε να μην είναι δυνατή η «πονηρή» παράκαμψη του μαθησιακού στόχου.

- Τεχνική πολυπλοκότητα ασφαλούς εκτέλεσης κώδικα: Σε αντίθεση με τα περισσότερα εκπαιδευτικά παιχνίδια, όπου η «είσοδος» του χρήστη είναι προκαθορισμένη (π.χ. πολλαπλή επιλογή, σύρε-άφησε), το "Reddy's Quest" καλεί τον χρήστη να γράψει πραγματικό, ελεύθερο κώδικα. Αυτό εισάγει σοβαρές τεχνικές προκλήσεις γύρω από την απομόνωση της εκτέλεσης κώδικα (sandboxing), την αποφυγή κακόβουλου ή προβληματικού κώδικα, καθώς και τη διαχείριση ενδεχόμενων ατέρμονων βρόχων, ζητήματα που δεν εμφανίζονται σε παιχνίδια με «κλειστές» φόρμες αλληλεπίδρασης.
- Ανομοιογένεια προηγούμενου επιπέδου γνώσεων: Οι μαθητές που θα χρησιμοποιήσουν το παιχνίδι μπορεί να έχουν πολύ διαφορετικό υπόβαθρο (από μηδενική έως κάποια προηγούμενη επαφή με προγραμματισμό), γεγονός που δυσκολεύει τον ενιαίο σχεδιασμό της καμπύλης δυσκολίας ώστε να παραμένει εντός της ζώνης επικείμενης ανάπτυξης για το σύνολο των χρηστών. Μια πολύ εύκολη πίστα μπορεί να προκαλέσει πλήξη σε πιο έμπειρους χρήστες, ενώ μια πολύ δύσκολη μπορεί να αποθαρρύνει έναν πλήρως αρχάριο.
- Κόστος ανάπτυξης και συντήρησης: Η δημιουργία ενός εκπαιδευτικού παιχνιδιού απαιτεί δεξιότητες τόσο στον τομέα της παιδαγωγικής όσο και στον τομέα της ανάπτυξης λογισμικού (σχεδιασμός επιπέδων, γραφικά, ήχος, αρχιτεκτονική κώδικα), κάτι που καθιστά την ανάπτυξη χρονοβόρα σε σχέση με πιο απλά, στατικά εκπαιδευτικά υλικά, ιδίως όταν αναλαμβάνεται από έναν και μόνο δημιουργό.
- Έλλειψη συστηματικής αξιολόγησης αποτελεσματικότητας: Τέλος, ένα ευρύτερο ζήτημα που υπάρχει είναι η σχετική έλλειψη αυστηρών εμπειρικών μελετών που να τεκμηριώνουν με σαφήνεια τη μαθησιακή αποτελεσματικότητα των εκπαιδευτικών παιχνιδιών έναντι παραδοσιακών μεθόδων, πέρα από την αύξηση του κινήτρου και της αφοσίωσης (Hamari et al., 2014). Το ζήτημα αυτό συνδέεται άμεσα με την ανάγκη μελλοντικής αξιολόγησης του "Reddy's Quest" σε πραγματικές συνθήκες χρήσης.

3 Το παιχνίδι “Reddy’s Quest”

Στο παρόν κεφάλαιο περιγράφεται αναλυτικά το παιχνίδι “Reddy’s Quest” που αναπτύχθηκε στο πλαίσιο της συγκεκριμένης πτυχιακής εργασίας. Παρουσιάζονται το πλαίσιο χρήση και η ηλικιακή ομάδα, οι τεχνολογίες που υλοποιήθηκαν, η αρχιτεκτονική του κώδικα, οι μηχανισμοί ασφαλούς εκτέλεσης και αξιολόγησης του κώδικα του χρήστη, όπως και τα ζητήματα που προέκυψαν κατά την ανάπτυξη. Επιπλέον, περιγράφεται η ροή καταστάσεων του παιχνιδιού, καθώς και οι απαιτήσεις εκτέλεσης της εφαρμογής.

3.1 Ηλικιακή ομάδα και πλαίσιο

Το “Reddy’s Quest” απευθύνεται σε μαθητές Γυμνασίου – Λυκείου (12-16 ετών), όπως και σε κάθε αρχάριο χρήστη χωρίς προηγούμενη εμπειρία στον προγραμματισμό. Η ηλικιακή αυτή ομάδα επιλέχθηκε επειδή συμπίπτει με την περίοδο κατά την οποία εισάγονται συνήθως τα πρώτα μαθήματα πληροφορικής στο ελληνικό εκπαιδευτικό σύστημα, γεγονός που καθιστά το παιχνίδι αξιοποιήσιμο τόσο ως συμπληρωματικό υλικό μέσα στη σχολική τάξη, υπό την καθοδήγηση εκπαιδευτικού, όσο και για αυτόνομη μελέτη στο σπίτι, χωρίς την ανάγκη συνεχούς επίβλεψης (Lister et al.,2004).

Το παιχνίδι δεν στοχεύει να αντικαταστήσει τη διδασκαλία ενός εκπαιδευτικού, αλλά να λειτουργήσει συμπληρωματικά προς αυτήν, καλύπτοντας συγκεκριμένα το στάδιο της πρώτης, εισαγωγικής επαφής με τις θεμελιώδεις έννοιες της Python (μεταβλητές, αριθμητικές πράξεις, συνθήκες, επαναλήψεις) (Resnick et al.,2009). Μπορεί να έχει δύο ρόλους: αφενός ως εργαλείο εξάσκησης μέσα στην τάξη, όπου ο εκπαιδευτικός έχει ήδη παρουσιάσει θεωρητικά μια έννοια και το παιχνίδι προσφέρει τον χώρο για ενεργή, πρακτική εφαρμογή της, αφετέρου ως αυτόνομο εργαλείο πρώτης επαφής, για μαθητές που θέλουν να δοκιμάσουν τον προγραμματισμό πριν διδαχθούν επίσημα το αντικείμενο, ή για ενηλίκους αρχάριους που αναζητούν ένα ευχάριστο σημείο εισόδου.

Η λειτουργία της εφαρμογής ως αυτόνομο πρόγραμμα επιφάνειας εργασίας (desktop application), χωρίς ανάγκη σύνδεσης στο διαδίκτυο, εγκατάστασης πρόσθετων εξαρτήσεων πέραν της Python και της Pygame, ή δημιουργίας λογαριασμού, ενισχύει περαιτέρω

αυτή τη χρησιμότητα και διευκολύνει τη χρήση της σε σχολικά εργαστήρια πληροφορικής όπου η πρόσβαση στο διαδίκτυο μπορεί να είναι περιορισμένη, αργή ή απαγορευμένη για λόγους ασφάλειας.

3.2 Σχεδιαστικό σκεπτικό

Η κεντρική σχεδιαστική επιλογή του "Reddy's Quest" ήταν ο συνδυασμός ελεύθερης σύνταξης κώδικα Python με ισχυρή αφηγηματική πλαισίωση, μια θέση ανάμεσα στα δύο «άκρα» που εντοπίστηκαν κατά την επισκόπηση υπαρχόντων εργαλείων (βλ. Κεφάλαιο 1, Ενότητα 1.2, και Κεφάλαιο 2, Ενότητα 2.2): από την μία τα εργαλεία οπτικού προγραμματισμού (π.χ. Scratch), από την άλλη οι πλατφόρμες άσκησης κώδικα χωρίς αφήγηση (Resnick et al., 2009). Ο χρήστης ενσαρκώνει τον χαρακτήρα Reddy, ο οποίος προσπαθεί να φτάσει στον «Πύργο της Python» για να βρει τον «Κρύσταλλο της Γνώσης», λύνοντας 8 προγραμματιστικά puzzles, όπου κάθε puzzle αντιστοιχεί σε μία αφηγηματική «αποστολή» (quest) με το δικό της σενάριο, ώστε η επίλυσή του να μη λειτουργεί ως απομονωμένη άσκηση αλλά ως φυσικό επόμενο βήμα της ιστορίας.

Η επιλογή αυτή δεν είναι απλώς αισθητική, βασίζεται στην αρχή, που αναλύθηκε θεωρητικά στο Κεφάλαιο 2, ότι η μάθηση είναι αποτελεσματικότερη όταν η ίδια η ενέργεια που προωθεί την πλοκή ταυτίζεται με τη μαθησιακή πράξη (Gee, 2003). Στο "Reddy's Quest", η συγγραφή σωστού κώδικα είναι η ίδια η κινητήρια δύναμη της ιστορίας: χωρίς σωστό κώδικα, ο Reddy δεν προχωράει.

Παράλληλα, η σταδιακή δυσκολία των οκτώ επιπέδων σχεδιάστηκε ώστε να ακολουθεί προσεγγιστικά τη λογική της Ζώνης Επικείμενης Ανάπτυξης και της Θεωρίας της Ροής (Csikszentmihalyi, 1990), με κάθε επίπεδο να εισάγει ακριβώς μία νέα έννοια πάνω σε όσες έχουν ήδη διδαχθεί, αποφεύγοντας απότομα άλματα δυσκολίας. Καθοριστική σχεδιαστική αρχή ήταν επίσης η ισοδυναμία μεταξύ επιτυχίας στο puzzle και κατάκτησης της έννοιας: ο έλεγχος της λύσης του χρήστη δεν βασίζεται μόνο στο τελικό αποτέλεσμα (output) αλλά και στη δομή του ίδιου του κώδικα. Έτσι αποφεύγεται το ενδεχόμενο ο χρήστης να «κερδίσει» ένα επίπεδο τυπώνοντας απευθείας το επιθυμητό αποτέλεσμα, χωρίς να χρησιμοποιήσει τη δομή που διδάσκει το συγκεκριμένο puzzle.

Τέλος, η άμεση οπτικοακουστική ανατροφοδότηση σχεδιάστηκε χωρίς καθυστέρηση ανάμεσα στην ενέργεια του χρήστη και την απόκριση του παιχνιδιού. Αυτό ενισχύει την αίσθηση αλληλεπίδρασης και μετατρέπει κάθε προσπάθεια σε άμεση μαθησιακή εμπειρία.

3.3 Τεχνολογίες υλοποίησης και τεκμηρίωση επιλογών

Για την υλοποίηση του "Reddy's Quest" αξιοποιήθηκαν αποκλειστικά η γλώσσα Python και η βιβλιοθήκη Pygame, μαζί με ενσωματωμένες βιβλιοθήκες της Standard library, χωρίς εξωτερικές εξαρτήσεις τρίτων. Η επιλογή αυτή ήταν συνειδητή, ώστε η εγκατάσταση και εκτέλεση του παιχνιδιού να παραμένει όσο το δυνατόν απλούστερη για τον τελικό χρήστη.

- Python 3.12: Επιλέχθηκε τόσο ως γλώσσα υλοποίησης της εφαρμογής όσο και ως γλώσσα διδασκαλίας, ώστε ο κώδικας που γράφει ο χρήστης μέσα στο παιχνίδι να είναι πραγματικός, εκτελέσιμος κώδικας. Η επιλογή της συγκεκριμένης έκδοσης έγινε λόγω της σταθερότητάς της, της πλήρους συμβατότητας με τις βιβλιοθήκες της standard library που χρησιμοποιήθηκαν και της ευρείας υποστήριξης από την κοινότητα (Python Software Foundation, 2026).
- Pygame: Χρησιμοποιήθηκε για τη δημιουργία ολόκληρου του γραφικού περιβάλλοντος (παράθυρο, σχήματα, κείμενο, ήχος, βρόχος κύριου προγράμματος). Η Pygame προτιμήθηκε λόγω της απλότητάς της, της ωριμότητάς της για την ανάπτυξη 2D desktop παιχνιδιών και της άμεσης συμβατότητας με καθαρό, διαδικαστικό Python κώδικα, χωρίς ενδιάμεσες τεχνολογίες ή περίπλοκα frameworks διαπαφής (Pygame Community, 2026).
- Ενσωματωμένη βιβλιοθήκη `ast` (Abstract Syntax Trees): Αποτελεί τον πυρήνα του μηχανισμού αξιολόγησης και ήταν καθοριστική επιλογή για την επίτευξη του βασικού παιδαγωγικού στόχου του παιχνιδιού. Επιτρέπει στο παιχνίδι να αναλύει τη δομή του κώδικα που γράφει ο χρήστης και όχι μόνο το τελικό του output, καθώς και να απαγορεύει προληπτικά ριψοκίνδυνες εντολές πριν καν εκτελεστεί ο κώδικας (Python Software Foundation, 2026).
- Ενσωματωμένες βιβλιοθήκες `io` και `contextlib`: Χρησιμοποιήθηκαν για την «παγίδευση» της εξόδου `print()` του κώδικα του χρήστη σε ένα αντικείμενο μνήμης (`io.StringIO`) μέσω του `contextlib.redirect_stdout`, ώστε το output να μπορεί να συγκριθεί προγραμματιστικά με τον στόχο του κάθε επιπέδου.
- Module `threading`: Χρησιμοποιήθηκε για την εκτέλεση του κώδικα του χρήστη σε ξεχωριστό νήμα με χρονικό όριο (`timeout 2` δευτερολέπτων), ώστε ενδε-

χόμενοι ατέρμονοι βρόχοι να μην «παγώνουν» ολόκληρη την εφαρμογή. Η επιλογή αυτή ήταν απαραίτητη επειδή ο χρήστης έχει πρόσβαση σε δομές επανάληψης (`while`, `for`) οι οποίες, ως άγνωστος και μη έμπιστος κώδικας, θα μπορούσαν εύκολα να οδηγήσουν σε μη τερματιζόμενη εκτέλεση.

- `Module array`: Χρησιμοποιήθηκε για την επιτόπου δημιουργία απλών ηχητικών εφέ μέσω μαθηματικής σύνθεσης κυματομορφών, αντί για την ενσωμάτωση έτοιμων ηχητικών αρχείων. Έτσι μειώθηκε το μέγεθος και οι εξωτερικές εξαρτήσεις της εφαρμογής, αν και με το κόστος περιορισμένης ποιότητας ήχου σε σχέση με προηχογραφημένα εφέ.

3.4 Ροή καταστάσεων και διεπαφή

Η συνολική λειτουργία του παιχνιδιού οργανώνεται σε ένα απλό αλλά σαφές σύστημα καταστάσεων (Garris et al., 2002). Το βασικό state του παιχνιδιού παίρνει τιμές όπως `intro`, `game` και `win`, ενώ κάθε επίπεδο λειτουργεί σε δύο εσωτερικές φάσεις, `dialogue` και `question`. Η μετάβαση από τη μία κατάσταση στην άλλη γίνεται με βάση την πρόοδο του χρήστη: αρχικά εμφανίζεται η εισαγωγική οθόνη, στη συνέχεια το παιχνίδι περνά στα `quest`, και μετά την ολοκλήρωση και των 8 επιπέδων μεταφέρεται στην τελική οθόνη νίκης.

Κάθε επίπεδο ξεκινά με τη φάση του διάλογου (`dialogue`), όπου προβάλλεται η αφηγηματική εισαγωγή της αποστολής (`quests`), και ακολουθεί η φάση της ερώτησης (`question`), όπου ο χρήστης αλληλεπιδρά με τον επεξεργαστή κώδικα (`editor`) και εκτελεί τον κώδικα. Όταν μια λύση αξιολογηθεί ως επιτυχής, ενεργοποιείται μία σύντομη παύση ανατροφοδότησης, έπειτα αυξάνεται ο δείκτης του επιπέδου, καθαρίζεται ο επεξεργαστής κώδικα και γίνεται επαναφορά της κατάστασης του βοηθήματος. Με αυτόν τον τρόπο, η ροή μεταξύ των αποστολών παραμένει ομαλή και προβλέψιμη, χωρίς να απαιτούνται πολύπλοκα μενού ή επιπλέον οθόνες.

Η διεπαφή του παιχνιδιού προσαρμόζεται δυναμικά στο περιεχόμενο κάθε επιπέδου μέσω της συνάρτησης `update_layout (level)`, η οποία υπολογίζει τα βασικά `pygame.Rect` στοιχεία ανάλογα με το μήκος της εκφώνησης, της αναμενόμενης εξόδου (`expected output`) και του βοηθήματος. Έτσι αποφεύγονται σταθερές συντεταγμένες και η διάταξη παραμένει λειτουργική ακόμη και όταν αλλάζει το κείμενο από επίπεδο σε επίπεδο.

Η πλοήγηση του χρήστη είναι πλήρως ενσωματωμένη στη διεπαφή μέσω πλήκτρων και ποντικιού. Ο επεξεργαστής κώδικα υποστηρίζει εισαγωγή και διαγραφή χαρακτήρων, μετακίνηση κέρσορα, αλλαγή γραμμής, καθώς και συντομεύσεις όπως Ctrl + Enter για εκτέλεση του κώδικα. Παράλληλα, η κύλιση του επεξεργαστή κώδικα (editor) και της αναμενόμενης εξόδου (output) γίνεται ανεξάρτητα, ώστε ο χρήστης να μπορεί να παρακολουθεί ταυτόχρονα τον κώδικα και τα αποτελέσματα.

3.5 Αφηγηματικό πλαίσιο και δομή επιπέδων

Το παιχνίδι είναι δομημένο σε 8 διαδοχικά επίπεδα (quests), καθένα από τα οποία εισάγει μία νέα, συγκεκριμένη έννοια προγραμματισμού και συνδέεται με ένα ξεχωριστό αφηγηματικό σκηνικό και οπτικό στοιχείο:

1. Επίπεδο 1 – Η Πύλη (print): Ο χρήστης καλείται να «στείλει το μυστικό μήνυμα» ώστε να ανοίξει μια πύλη, τυπώνοντας ακριβώς το κείμενο "Hello, Reddy!". Ως πρώτο επίπεδο, εισάγει τη θεμελιώδη έννοια της εξόδου, χωρίς καμία επιπλέον πολυπλοκότητα (μεταβλητές, πράξεις), ώστε ο χρήστης να εξοικειωθεί πρώτα με τη διεπαφή του επεξεργαστή κώδικα (editor) και το κουμπί RUN.
2. Επίπεδο 2 – Ο Κρύσταλλος (μεταβλητές): Ο χρήστης αποθηκεύει το όνομα της γλώσσας σε μια μεταβλητή (`name = "Python"`) και το τυπώνει. Το επίπεδο αυτό εισάγει την έννοια της ανάθεσης τιμής και της αναφοράς σε μεταβλητή, χτίζοντας άμεσα πάνω στη γνώση του print από το προηγούμενο επίπεδο.
3. Επίπεδο 3 – Η Κλειδαριά (αριθμητικές πράξεις): Ο χρήστης υπολογίζει και τυπώνει το αποτέλεσμα μιας πράξης ($12 + 8$) για να «ξεκλειδώσει» μια κλειδαριά, εισάγοντας τις βασικές αριθμητικές πράξεις.
4. Επίπεδο 4 – Η Ξύλινη Πύλη (if / else): Ο χρήστης ορίζει μια μεταβλητή βαθμολογίας (`score = 7`) και, ανάλογα με την τιμή της, τυπώνει "PASS" ή "FAIL". Πρόκειται για το πρώτο επίπεδο που εισάγει λήψη απόφασης (δομή ελέγχου), σηματοδοτώντας αισθητή αύξηση δυσκολίας σε σχέση με τα προηγούμενα.
5. Επίπεδο 5 – Ο Δράκος (if / elif / else): Εισάγεται τριπλή διακλάδωση, με τον χρήστη να καθορίζει το μήνυμα ("HOT"/"WARM"/"COLD") ανάλογα με μια τιμή θερμοκρασίας, χτίζοντας πάνω στη λογική if/else του προηγούμενου επιπέδου με προσθήκη ενδιάμεσης συνθήκης.

6. Επίπεδο 6 – Ο Μηχανισμός (`while`): Ο χρήστης χρησιμοποιεί βρόχο `while` για να τυπώσει τους αριθμούς 1 έως 3, εισάγοντας για πρώτη φορά την έννοια της επανάληψης υπό συνθήκη.
7. Επίπεδο 7 – Οι Πύργοι (`for`): Ο χρήστης χρησιμοποιεί βρόχο `for` με `range` για να τυπώσει τον χαρακτήρα "!" τέσσερις φορές, εισάγοντας τη μετρήσιμη επανάληψη ως εναλλακτική στη λογική του `while`.
8. Quest 8 – Η Γέφυρα (συνδυασμός εννοιών): Το τελευταίο επίπεδο λειτουργεί ως «τελικό αφεντικό» (`final boss`), απαιτώντας συνδυασμό `for`, `range` και `if`, ώστε ο χρήστης να «χτίσει» μια γέφυρα τυπώνοντας εναλλάξ "Reddy" (για άρτιους αριθμούς) και "-" (για μονούς), από το 1 έως το 5, συνθέτοντας όλες τις έννοιες που διδάχθηκαν στα προηγούμενα επτά επίπεδα.

Κάθε επίπεδο ακολουθεί την ίδια δομή αλληλεπίδρασης σε δύο διαδοχικές φάσεις. Αρχικά εμφανίζεται μια σύντομη σκηνή διαλόγου (`phase = "dialogue"`), όπου ένα «σύννεφο ομιλίας» παρουσιάζει το αφηγηματικό σκηνικό του quest και ο χρήστης προχωρά με το πλήκτρο `SPACE`. Στη συνέχεια ο χρήστης μεταβαίνει στη φάση επίλυσης (`phase = "question"`), όπου εμφανίζονται η εκφώνηση του puzzle, το αναμενόμενο αποτέλεσμα (`expected output`) σε ξεχωριστό πλαίσιο, ένας απλός επεξεργαστής κώδικα (`editor`), ένα πλαίσιο εξόδου (`output`) που δείχνει το πραγματικό αποτέλεσμα του κώδικα μετά από κάθε εκτέλεση, το κουμπί `RUN`, και ένα αντίστοιχο οπτικό στοιχείο ανά επίπεδο (π.χ. πύλη, κρύσταλλος, κλειδαριά, δράκος, πύργοι) που ενισχύει οπτικά το σενάριο.

Η συνολική πρόοδος του χρήστη (τρέχον επίπεδο, αστέρια ανά quest, συνολικά αστέρια) παραμένει ορατή σε όλη τη διάρκεια του παιχνιδιού μέσω μιας γραμμής προόδου (`progress bar`) στο επάνω μέρος της οθόνης, ενώ η ολοκλήρωση όλων των επιπέδων οδηγεί σε μια τελική οθόνη νίκης (`state = "win"`) με συνοπτική παρουσίαση της συνολικής συλλογής αστεριών.

3.6 Αρχιτεκτονική του κώδικα

Ο κώδικας του "Reddy's Quest" είναι οργανωμένος σε ένα ενιαίο αρχείο Python (περίπου 1700 γραμμές), το οποίο όμως εσωτερικά χωρίζεται με σαφείς και σχολιασμένες επικεφαλίδες σε δεκαέξι λειτουργικές ενότητες, οργανωμένες κατά προσέγγιση με τη

σειρά που εκτελούνται λογικά κατά τη διάρκεια της εφαρμογής. Πρώτα η ρύθμιση παραθύρου, στη συνέχεια οι μηχανισμοί ασφάλειας και αξιολόγησης, έπειτα η σχεδίαση, και τέλος ο κύριος βρόχος). Η επιλογή ενός single-file αρχείου, αντί για διαχωρισμό σε πολλαπλά modules, ήταν συνειδητή για ένα έργο αυτής της κλίμακας: απλοποιεί τη διανομή του παιχνιδιού (ένα μόνο αρχείο .py) και τη γρήγορη πλοήγηση μέσα στον κώδικα μέσω των επικεφαλίδων ενότητας, με το κόστος μειωμένης δυνατότητας επαναχρησιμοποίησης επιμέρους τμημάτων σε άλλο πρόγραμμα.

Οι δεκαέξι ενότητες, με τη σειρά εμφάνισής τους στο αρχείο, είναι οι εξής:

- WINDOW: Αρχικοποίηση της Pygame (`pygame.init()`, `pygame.mixer.init()`) και δημιουργία του κύριου παραθύρου της εφαρμογής, με σταθερές διαστάσεις 900×680 pixel και τίτλο παραθύρου.
- COLORS: Ορισμός της πλήρους παλέτας χρωμάτων της εφαρμογής ως σταθερές (π.χ. `OUTPUT_BG`, `EDITOR_BG`, `MAP_DONE`, `MAP_LOCKED`) και όλων των γραμματοσειρών που χρησιμοποιούνται (π.χ. `code_font` σε Consolas για τον επεξεργαστή κώδικα, ώστε ο κώδικας να εμφανίζεται με σταθερό πλάτος χαρακτήρα όπως σε πραγματικό περιβάλλον προγραμματισμού, και ξεχωριστές γραμματοσειρές `title_font`, `quest_font`, `small_font` για τα υπόλοιπα στοιχεία της διεπαφής).
- STATE: Κεντρική διαχείριση της κατάστασης του παιχνιδιού μέσω global μεταβλητών: το γενικό state της εφαρμογής (`intro/game/win`), η τρέχουσα κατάσταση (`phase`) εντός ενός επιπέδου (`dialogue/question`), ο δείκτης τρέχοντος επιπέδου (`level_index`), το περιεχόμενο του editor ως λίστα γραμμών κειμένου (`user_lines`) μαζί με τη θέση του κέρσορα (`cursor_line`, `cursor_col`) και την κύλιση, η έξοδος και η ανατροφοδότηση (`feedback`) της τελευταίας εκτέλεσης, ο μετρητής λανθασμένων προσπαθειών, η κατάσταση του βοηθήματος (ξεκλειδωμένο/ορατό/χρησιμοποιημένο), ο πίνακας αστεριών ανά επίπεδο (`level_stars`) και οι μεταβλητές που οδηγούν τα κινούμενα γραφικά (`shake`, `bounce`, `flash`, `confetti`). Η προσέγγιση αυτή, αν και βασισμένη σε global μεταβλητές αντί για αντικειμενοστραφή ενθυλάκωση, ταιριάζει στην κλίμακα ενός single - file εκπαιδευτικού παιχνιδιού.

- **LAYOUT:** Ορισμός βασικών σταθερών διάταξης (π.χ. ύψος γραμμής επεξεργαστή κώδικα, αριθμός ορατών γραμμών, περιθώρια ανάμεσα σε πλαίσια) και αρχικοποίηση των `pygame.Rect` που καθορίζουν τη θέση και το μέγεθος κάθε βασικού στοιχείου της διεπαφής (πάνελ ερώτησης, πάνελ αναμενόμενου αποτελέσματος, επεξεργαστή κώδικα, πάνελ εξόδου, κουμπί RUN, λυχνάρι βοήθηματος).
- **SOUNDS:** Προγραμματιστική (procedural) δημιουργία απλών ηχητικών εφέ, χωρίς εξωτερικά αρχεία ήχου. Οι συναρτήσεις `generate_tone()` και `generate_chord()` συνθέτουν ωμά δείγματα ήχου (raw audio samples) μέσω μαθηματικών ημιτονοειδών κυματομορφών με φθίνουσα περιβάλλουσα ενέργειας (envelope), τα οποία μετατρέπονται απευθείας σε αντικείμενα `pygame.mixer.Sound` μέσω του `module array`. Τα τέσσερα παραγόμενα ηχητικά εφέ (`run`, `error`, `success`, `star`) αποθηκεύονται στο λεξικό `SOUNDS` για άμεση αναπαραγωγή μέσω της `play_sound()`, με προστασία (`try/except pygame.error`) για την περίπτωση που ο ήχος δεν είναι διαθέσιμος στο σύστημα.
- **PARTICLES:** Απλό σύστημα σωματιδίων (particle system) για οπτικά εφέ εορτασμού (`confetti`). Η `spawn_particles()` δημιουργεί έναν αριθμό σωματιδίων σε συγκεκριμένο σημείο, το καθένα με τυχαία αρχική ταχύτητα, χρώμα, μέγεθος και διάρκεια ζωής. Η `update_particles()` ενημερώνει σε κάθε καρέ τη θέση κάθε σωματιδίου εφαρμόζοντας απλή προσομοίωση βαρύτητας και αφαιρεί τα σωματίδια των οποίων η διάρκεια ζωής έχει μηδενιστεί. Η `draw_particles()` τα σχεδιάζει με φθίνουσα διαφάνεια (`alpha`) ανάλογη της εναπομένουσας διάρκειας ζωής τους, δίνοντας την αίσθηση ότι «σβήνουν» σταδιακά.
- **LAYOUT HELPERS:** Ένα σύνολο συναρτήσεων για δυναμική, προσαρμοστική διάταξη (`responsive layout`), το οποίο ξεχωρίζει ως ένα από τα πιο τεχνικά ενδιαφέροντα σημεία της υλοποίησης. Οι `measure_wrapped_lines()` και `panel_height_for_lines()` υπολογίζουν πόσες γραμμές θα χρειαστεί ένα κείμενο μετά από αναδίπλωση λέξεων (`word wrapping`) σε συγκεκριμένο πλάτος, και άρα το απαιτούμενο ύψος ενός πάνελ. Η κεντρική συνάρτηση `update_layout(level)`, που καλείται κάθε φορά που αλλάζει επίπεδο, υπολογίζει εκ νέου τις θέσεις και τα ύψη όλων των πάνελ (ερώτηση, αναμενόμενο αποτέλεσμα, επεξεργαστή κώδικα, έξοδος) ανάλογα με το πραγματικό μήκος του

κειμένου του συγκεκριμένου επιπέδου, χρησιμοποιώντας μια επαναληπτική διαδικασία συρρίκνωσης (iterative shrinking): ξεκινά δοκιμάζοντας τον μέγιστο αριθμό ορατών γραμμών για τον editor και, αν το σύνολο των πάνελ δεν χωράει εντός του ύψους της οθόνης, μειώνει σταδιακά πρώτα τις γραμμές του επεξεργαστή κώδικα και έπειτα το ύψος του πάνελ εξόδου, μέχρι να επιτευχθεί διάταξη που χωράει πλήρως στο παράθυρο. Ο μηχανισμός αυτός επιτρέπει στο ίδιο σταθερό παράθυρο 900×680 να φιλοξετεί σωστά τόσο πολύ σύντομα puzzles (π.χ. Quest 1) όσο και πιο εκτενή, με μεγαλύτερη εκφώνηση ή αναμενόμενο αποτέλεσμα πολλαπλών γραμμών (π.χ. Quest 8), χωρίς επικαλύψεις στοιχείων της διεπαφής.

- SANDBOX / VALIDATION / EXECUTION: Ο πυρήνας της ασφαλούς εκτέλεσης, ο στατικός έλεγχος του AST και η εκτέλεση με χρονικό όριο σε ξεχωριστό νήμα (αναλύεται λεπτομερώς στην Ενότητα 3.7), καθώς και ο μηχανισμός αξιολόγησης του κώδικα του χρήστη μέσω συναρτήσεων ελέγχου ανά επίπεδο(αναλύονται στην Ενότητα 3.8).
- LEVELS: Δομή δεδομένων (λίστα λεξικών επιπέδων, μία εγγραφή ανά επίπεδο) που περιγράφει τα 8 επίπεδα: έννοια (concept), τίτλο αποστολής, κείμενο διαλόγου, εκφώνηση ερώτησης puzzle (question), αναμενόμενο αποτέλεσμα (expected), ενδεικτική λύση για το hint, αναφορά στην αντίστοιχη συνάρτηση ελέγχου (check) και αναγνωριστικό οπτικού στοιχείου (visual, π.χ. "gate", "dragon", "bridge"). Ο διαχωρισμός του περιεχομένου των επιπέδων από τη μηχανική του παιχνιδιού σε αυτή τη δομή δεδομένων καθιστά σχετικά απλή την προσθήκη ή τροποποίηση επιπέδων στο μέλλον, χωρίς αλλαγές στην υπόλοιπη λογική. Ενδεικτικά, η εγγραφή του πρώτου επιπέδου παρατίθεται στην Εικόνα 1. Η πλήρης λίστα και των οκτώ επιπέδων παρατίθεται στο Παράρτημα Α.1.

```
553 levels = [  
554     {  
555         "concept": "print",  
556         "quest_title": "Quest 1: Η Πύλη",  
557         "dialogue": "Ο πρώτος φρουρός δεν αναγνωρίζει τον Reddy. Η πύλη θα  
558             ανοίξει μόνο αν εμφανιστεί το σωστό μήνυμα.",  
559         "question": "PUZZLE: Στείλε το μυστικό μήνυμα στην πόρτα.\nΠρέπει να  
560             εμφανιστεί ακριβώς:\nHello, Reddy!",  
561         "expected": "Hello, Reddy!",  
562         "hint": "print('Hello, Reddy!')",  
563         "check": check_level_1,  
564         "visual": "gate",  
565     },
```

Εικόνα 1: Η εγγραφή του πρώτου επιπέδου (Quest 1) στη δομή δεδομένων levels.

- **SCORING:** Η συνάρτηση `calculate_stars()` που υπολογίζει τα αστέρια ενός επιπέδου βάσει προσπαθειών και βοηθημάτων, και η `total_stars()` που αθροίζει τα αστέρια όλων των επιπέδων (βλ. Ενότητα 3.9).
- **EDITOR:** Υλοποίηση ενός πλήρους, πολυγραμμικού επεξεργαστή κειμένου εντός της Pygame από την αρχή, αφού η βιβλιοθήκη δεν προσφέρει έτοιμο στοιχείο πεδίου κειμένου. Περιλαμβάνει διαχείριση της θέσης του κέρσορα σε δύο διαστάσεις (γραμμή/στήλη), εισαγωγή και διαγραφή χαρακτήρων, χειρισμό ειδικών πλήκτρων (Enter για νέα γραμμή, Backspace/Delete, Tab), οριζόντια και κάθετη κύλιση (`editor_scroll`) όταν ο κώδικας ξεπερνά τις ορατές γραμμές ή στήλες, καθώς και ένα οπτικό κέρσορα που αναβοσβήνει (`cursor_blink_time`) για οπτική ένδειξη της θέσης εισαγωγής.
- **ANIMATIONS:** Οι συναρτήσεις `trigger_error_fx()` και `trigger_success_fx()`, οι οποίες ενεργοποιούν συνδυαστικά τα οπτικοακουστικά εφέ ανατροφοδότησης (screen shake, flash overlay, bounce του Reddy, σωματίδια, ήχος) ανάλογα με το αν ο κώδικας ήταν λανθασμένος ή σωστός, και η `update_animations(dt)`, που ενημερώνει σε κάθε καρέ τις παραμέτρους αυτών των εφέ (π.χ. σταδιακή μείωση της έντασης της δόνησης (shake) και της διαφάνειας της φωτεινής αναλαμπής (flash) με την πάροδο του χρόνου) βάσει του χρόνου που έχει περάσει από το προηγούμενο καρέ (delta time).
- **DRAW - UI:** Το μεγαλύτερο σε έκταση τμήμα του κώδικα, με συναρτήσεις σχεδίασης για κάθε οπτικό στοιχείο της διεπαφής: γενικές βοηθητικές συναρτήσεις κειμένου με αναδίπλωση (`draw_wrapped_multiline`, `draw_multiline_in_rect`), τη γραμμή προόδου (`draw_progress_bar`) στο πάνω μέρος της οθόνης, τη σχεδίαση του ίδιου του χαρακτήρα Reddy μέσω βασικών γεωμετρικών σχημάτων (κύκλοι και τόξα για σώμα, μάτια και στόμα, με διαφορετική έκφραση ανάλογα με το αν είναι «χαρούμενος»), τη σχεδίαση των αστεριών βαθμολόγησης (`draw_stars`, μέσω υπολογισμού σημείων πολυγώνου αστεριού), και τη `draw_puzzle_visual()`, η οποία σχεδιάζει ένα διαφορετικό, χειροποίητο γραφικό ανά τύπο επιπέδου (π.χ. πύλη με ένδειξη "DENIED", κρύσταλλος, κλειδαριά με την πράξη "12+8=?", δράκος με φλόγα, γέφυρα) βάσει του πεδίου `visual` κάθε επιπέδου.

- **RUN CODE:** Η συνάρτηση `try_run_code()`, η οποία λειτουργεί ως το σημείο σύνδεσης ανάμεσα στον μηχανισμό εκτέλεσης/ελέγχου (`SANDBOX / VALIDATION / EXECUTION`) και στην υπόλοιπη κατάσταση του παιχνιδιού: καλεί την εκτέλεση του κώδικα, ερμηνεύει το αποτέλεσμα, ενημερώνει τα μηνύματα ανατροφοδότησης, τον μετρητή λανθασμένων προσπαθειών, την κατάσταση του βοηθήματος και τα αστέρια, και ενεργοποιεί τα αντίστοιχα οπτικοακουστικά εφέ μέσω των συναρτήσεων της ενότητας `ANIMATIONS`.
- **MAIN LOOP:** Ο κύριος βρόχος της εφαρμογής, ρυθμισμένος σε σταθερό ρυθμό 60 καρέ ανά δευτερόλεπτο (`clock.tick(60)`). Σε κάθε επανάληψη: υπολογίζεται ο χρόνος που πέρασε από το προηγούμενο καρέ (`dt`), ενημερώνονται τα κινούμενα γραφικά και τα σωματίδια, καθαρίζεται η οθόνη, και σχεδιάζεται εκ νέου το σύνολο της διεπαφής ανάλογα με το τρέχον `state` (`intro/game/win`) και `phase` (`dialogue/question`), μέσα από μια αλυσίδα υπό-συνθηκών `if/elif`. Παράλληλα, ο βρόχος διαχειρίζεται όλα τα συμβάντα εισόδου μέσω `pygame.event.get()`: πλοήγηση και επεξεργασία στον επεξεργαστή κώδικα από το πληκτρολόγιο, κλικ ποντικιού στο κουμπί `RUN`, στο λυχνάρι βοηθήματος ή στα κουμπιά πλοήγησης, και κύλιση με τη ρόδα του ποντικιού στα πάνελ εξόδου/επεξεργαστή κώδικα.

3.7 Μηχανισμός ασφαλούς εκτέλεσης κώδικα (Sandbox)

Δεδομένου ότι ο χρήστης γράφει ελεύθερο κώδικα Python και όχι προκαθορισμένες επιλογές, η ασφαλής εκτέλεσή του αποτέλεσε κρίσιμο ζήτημα σχεδιασμού, τόσο για την προστασία του υπολογιστή στον οποίο τρέχει το παιχνίδι όσο και για τη σταθερότητα της ίδιας της εφαρμογής. Η προσέγγιση που ακολουθήθηκε βασίζεται σε τέσσερα συμπληρωματικά επίπεδα άμυνας.

Πρώτον, στατικός έλεγχος πριν την εκτέλεση: πριν καν εκτελεστεί ο κώδικας, αναλύεται το Abstract Syntax Tree (AST) του, μέσω της συνάρτησης `validate_ast()`. Ο έλεγχος αυτός διατρέχει όλους τους κόμβους του AST (μέσω `ast.walk`) και απορρίπτει κώδικα που περιέχει εντολές `import` ή `from ... import`, χρήση ονομάτων από μια ρητή «μαύρη λίστα» (`BANNED_NAMES: eval, exec, compile, open, import, globals, locals, vars, exit, quit`), πρόσβαση σε `attributes` αντικειμένων, καθώς

και ατέρμονους βρόχους της μορφής `while True`. Ο έλεγχος αυτός γίνεται πριν από οποιαδήποτε εκτέλεση, οπότε ο κώδικας απορρίπτεται με σαφές μήνυμα χωρίς καν να τρέξει. Η συνάρτηση `validate_ast` που υλοποιεί αυτόν τον έλεγχο παρατίθεται στην Εικόνα 2.

```
856 def validate_ast(tree):
857     for node in ast.walk(tree):
858         if isinstance(node, (ast.Import, ast.ImportFrom)):
859             return False, " Δεν επιτρέπεται η χρήση import."
860         if isinstance(node, ast.Name) and node.id in BANNED_NAMES:
861             return False, f" Η εντολή '{node.id}' δεν επιτρέπεται."
862         if isinstance(node, ast.Attribute):
863             return False, " Δεν επιτρέπεται πρόσβαση σε εσωτερικά αντικείμενα."
864         if isinstance(node, ast.While):
865             if isinstance(node.test, ast.Constant) and node.test.value is True:
866                 return False, " Δεν επιτρέπεται while True."
867         if isinstance(node, (ast.Break, ast.Continue)):
868             return False, " Δεν επιτρέπεται break/continue."
869         if isinstance(node, ast.Call):
870             if isinstance(node.func, ast.Name) and node.func.id in BANNED_NAMES:
871                 return False, f" Η εντολή '{node.func.id}' δεν επιτρέπεται."
872     return True, ""
```

Εικόνα 2: Η συνάρτηση `validate_ast()`, που διατρέχει το AST του κώδικα του χρήστη και απορρίπτει απαγορευμένες εντολές πριν την εκτέλεση.

Δεύτερον, περιορισμένο περιβάλλον εκτέλεσης: όταν ο κώδικας κριθεί ασφαλής στατικά, εκτελείται μέσω της Python `exec()` αλλά με ένα ρητά περιορισμένο λεξικό `builtins` (`SAFE_BUILTINS = {"print", "range", "len", "int", "float", "str", "bool"}`), αντί για το πλήρες σύνολο ενσωματωμένων συναρτήσεων της Python. Έτσι, ακόμη και αν κάποιος έλεγχος στατικής ανάλυσης παρακαμφθεί απρόβλεπτα, ο εκτελούμενος κώδικας δεν έχει πρόσβαση σε ευαίσθητες λειτουργίες.

Τρίτον, εκτέλεση με χρονικό όριο σε ξεχωριστό νήμα: ο κώδικας εκτελείται σε ξεχωριστό thread (μέσω `threading.Thread`), το οποίο η κύρια εφαρμογή περιμένει (`join`) για ανώτατο χρόνο 2 δευτερολέπτων (`EXECUTION_TIMEOUT`). Αν ο κώδικας δεν έχει ολοκληρωθεί ως τότε, το κύριο πρόγραμμα ενημερώνει τον χρήστη για πιθανό ατέρμονο βρόχο και συνεχίζει κανονικά τη λειτουργία του, χωρίς να «παγώσει». Ένα `threading.Lock` διασφαλίζει επιπλέον ότι δεν μπορούν να ξεκινήσουν παράλληλες εκτελέσεις κώδικα από διαδοχικά πατήματα του κουμπιού RUN. Επιπλέον, εφαρμόζονται όρια στο μέγεθος του κώδικα (έως 1500 χαρακτήρες και 80 γραμμές), ώστε να αποτρέπονται εκφυλισμένες περιπτώσεις υπερβολικά μεγάλου κώδικα.

Τέταρτον, απομόνωση της εξόδου: η έξοδος (`stdout`) του κώδικα παγιδεύεται μέσω `io.StringIO` και `contextlib.redirect_stdout`, ώστε να μη γράφεται απευθείας εκτός ελέγχου, αλλά να συλλέγεται σε ένα αντικείμενο μνήμης και να συγκρίνεται

προγραμματιστικά με τον στόχο κάθε επιπέδου, χωρίς να επηρεάζει την υπόλοιπη εφαρμογή. Ο συνδυασμός των τεσσάρων αυτών μηχανισμών καλύπτει τα πιο συνήθη ρίσκα που μπορεί να προκύψουν από τον κώδικα ενός αρχάριου μαθητή, χωρίς ωστόσο να ισοδυναμεί με πλήρες sandboxing επιπέδου λειτουργικού συστήματος, όπως συζητείται στην Ενότητα 3.9.

Ο πλήρης κώδικας του μηχανισμού απομόνωσης της εκτέλεσης κώδικα (στατικός έλεγχος, περιορισμένα builtins, εκτέλεση με timeout) παρατίθεται στο Παράρτημα Α.2.

3.8 Μηχανισμός αξιολόγησης του κώδικα χρήστη

Η αξιολόγηση της ορθότητας μιας λύσης δεν βασίζεται μόνο στη σύγκριση του output με τον στόχο του επιπέδου, αλλά και στην ανάλυση της δομής του κώδικα μέσω του AST, ώστε η επιτυχία σε ένα puzzle να αντιστοιχεί πράγματι στη χρήση της έννοιας που διδάσκει το συγκεκριμένο επίπεδο και όχι σε τυχαία ή «πονηρή» λύση. Για τον σκοπό αυτό αναπτύχθηκαν οκτώ βοηθητικές συναρτήσεις ανάλυσης δομής: η `uses_print()` εντοπίζει κλήσεις της `print()`, η `assigns_variable()` ελέγχει αν έχει γίνει ανάθεση τιμής σε συγκεκριμένη μεταβλητή, η `prints_variable()` ελέγχει αν μια συγκεκριμένη μεταβλητή τυπώνεται, η `has_arithmetic()` εντοπίζει αριθμητικές πράξεις, η `has_if_else()` και η `has_if_elif_else()` εντοπίζουν αντίστοιχα απλή και πολλαπλή διακλάδωση, ενώ η `has_while()` και η `has_for_with_range()` εντοπίζουν τις δύο βασικές δομές επανάληψης. Καθεμία από αυτές διατρέχει το AST αναζητώντας συγκεκριμένους τύπους κόμβων.

Κάθε επίπεδο διαθέτει τη δική του συνάρτηση ελέγχου (π.χ. `check_level_4` για το Quest 4), η οποία συνδυάζει δύο κριτήρια: πρώτον, τη δομική επιβεβαίωση ότι χρησιμοποιήθηκε η απαιτούμενη προγραμματιστική δομή (π.χ. στο Quest 4, ότι υπάρχει πράγματι `if/else`. Στο Quest 8, ότι υπάρχουν ταυτόχρονα `for` με `range` και `if` μέσα στο σώμα του βρόχου), δεύτερον, τη σύγκριση του κανονικοποιημένου output του κώδικα, μέσω της `normalize_output()` (που μετατρέπει το κείμενο εξόδου σε λίστα γραμμών, αφαιρώντας κενά τέλους γραμμής), με το ακριβές αναμενόμενο αποτέλεσμα του επιπέδου. Μόνο αν πληρούνται και τα δύο κριτήρια, το puzzle θεωρείται λυμένο, διαφορετικά, ο χρήστης λαμβάνει συγκεκριμένο, στοχευμένο μήνυμα ανατροφοδότησης (π.χ. «Χρησιμοποίησε `if` και `else`», «Το output σου δεν ταιριάζει με τον στόχο»), το οποίο τον καθοδηγεί προς τη λύση χωρίς να του την αποκαλύπτει άμεσα.

Για παράδειγμα, στο Quest 8 (η πιο απαιτητική συνάρτηση ελέγχου, `check_level_8`), ελέγχεται διαδοχικά αν υπάρχει `for` loop με `range`, αν υπάρχει τουλάχιστον ένα `if` οπουδήποτε στο δέντρο, αν χρησιμοποιείται `print`, και τέλος αν το `output` ταιριάζει ακριβώς με την ακολουθία `["-", "Reddy", "-", "Reddy", "-"]`, μια αλληλουχία ελέγχων που εξασφαλίζει πως ο χρήστης δεν μπορεί να πετύχει το επίπεδο παρακάμπτοντας κάποια από τις τρεις έννοιες (βρόχο, συνθήκη, έξοδο) που το επίπεδο επιδιώκει να συνδυάσει. Ο πλήρης κώδικας της συνάρτησης παρατίθεται στην Εικόνα 3.

```
536 def check_level_8(code, tree, out):
537     if not has_for_with_range(tree):
538         return False, "Χρησιμοποίησε for ... in range(...)"
539     if not any(isinstance(n, ast.If) for n in ast.walk(tree)):
540         return False, "Χρησιμοποίησε if μέσα στο loop"
541     if not uses_print(tree):
542         return False, "Χρησιμοποίησε print(...)"
543     expected = ["-", "Reddy", "-", "Reddy", "-"]
544     if normalize_output(out) != expected:
545         return False, "Το output σου δεν ταιριάζει με τον στόχο"
546     return True, ""
```

Εικόνα 3: Η συνάρτηση ελέγχου `check_level_8()`, που επιβεβαιώνει διαδοχικά τη χρήση `for` με `range`, την ύπαρξη `if`, τη χρήση `print`, και τέλος την ακρίβεια του `output`.

Στη διαδικασία εκτέλεσης προβλέπεται επίσης εξειδικευμένος χειρισμός σφαλμάτων χρόνου εκτέλεσης (`NameError`, `TypeError`, `ZeroDivisionError`, `IndentationError`, καθώς και γενικά `SyntaxError` κατά το parsing), με μηνύματα προσαρμοσμένα ώστε να είναι κατανοητά σε έναν αρχάριο μαθητή (π.χ. «Μεταβλητή που δεν υπάρχει», «Λάθος τύπος δεδομένων», «Δεν μπορείς να διαιρέσεις με το μηδέν»), αντί να εμφανίζεται απευθείας το ακατέργαστο, τεχνικό μήνυμα εξαίρεσης της Python, το οποίο συχνά είναι δυσνόητο για κάποιον χωρίς προηγούμενη εμπειρία.

3.9 Σύστημα βαθμολόγησης και βοηθημάτων

Η πρόοδος του χρήστη ενισχύεται μέσω ενός συστήματος τριών αστεριών ανά επίπεδο, το οποίο υπολογίζεται από τη συνάρτηση `calculate_stars()` βάσει δύο παραγόντων: του αριθμού λανθασμένων προσπαθειών (`wrong_attempts`) στο τρέχον επίπεδο και της χρήσης ή μη του βοηθήματος. Συγκεκριμένα, ο χρήστης λαμβάνει τα τρία αστέρια μόνο αν πετύχει από την πρώτη προσπάθεια χωρίς να χρησιμοποιήσει βοήθημα, δύο αστέρια αν χρειάστηκε περισσότερες από μία προσπάθειες αλλά χωρίς βοήθημα, και

ένα αστέρι αν χρησιμοποίησε το βοήθημα, ανεξαρτήτως αριθμού προσπαθειών. Η βαθμολόγηση αυτή λειτουργεί ως έμμεσος δείκτης βαθύτητας κατανόησης, ανταμείβοντας περισσότερο την ανεξάρτητη, επιτυχημένη προσπάθεια. Το σύνολο των αστεριών ανά επίπεδο (`level_stars`) διατηρεί πάντα την υψηλότερη βαθμολογία που έχει πετύχει ο χρήστης σε τυχόν επαναλαμβανόμενες προσπάθειες του ίδιου επιπέδου, ενώ το άθροισμά τους (`total_stars()`) παραμένει ορατό σε όλη τη διάρκεια του παιχνιδιού, ενισχύοντας διαρκώς την αίσθηση προόδου, και παρουσιάζεται συγκεντρωτικά και στην τελική οθόνη νίκης.

Το σύστημα Hints λειτουργεί ως προαιρετικό βοήθημα, όχι ως άμεσα διαθέσιμη επιλογή: ξεκλειδώνεται (`hint_unlocked = True`) μόνο μετά από δύο συνεχόμενες λανθασμένες προσπάθειες στο ίδιο επίπεδο, ώστε να ενθαρρύνεται η αυτόνομη προσπάθεια πριν ο χρήστης καταφύγει σε βοήθεια. Όταν είναι διαθέσιμο, εμφανίζεται ως ένα διαδραστικό «λυχνάρι» στη διεπαφή, το οποίο ανάβει οπτικά (*pulsing animation*) για να τραβήξει την προσοχή του χρήστη, και το οποίο ο χρήστης μπορεί να πατήσει για να εμφανίσει ή να κρύψει μια ενδεικτική λύση του τρέχοντος puzzle. Η κατάσταση αυτή (πόσες φορές έγινε λάθος, αν έχει ξεκλειδωθεί ή χρησιμοποιηθεί το βοήθημα) μηδενίζεται αυτόματα στην αρχή κάθε νέου επιπέδου, ώστε η βαθμολόγηση κάθε quest να είναι ανεξάρτητη από τις προηγούμενες.

Παράλληλα με το σύστημα αστεριών, κάθε προσπάθεια εκτέλεσης συνοδεύεται από άμεση πολυαισθητηριακή ανατροφοδότηση: σε επιτυχία, ενεργοποιείται ηχητικό εφέ συγχορδίας, αναπήδηση του χαρακτήρα Reddy, πράσινη φωτεινή επικάλυψη και σωματίδια εορτασμού σε χρυσό και πράσινο χρώμα· σε αποτυχία, ενεργοποιείται διαφορετικό ηχητικό εφέ, «τίναγμα» της οθόνης και κόκκινη φωτεινή επικάλυψη. Ο συνδυασμός αυτός αποσκοπεί στο να κλείνει άμεσα, σε κάθε πάτημα του κουμπιού RUN, τον κύκλο ανατροφοδότησης που περιγράφηκε θεωρητικά στο Κεφάλαιο 2 (Garris et al., 2002), χωρίς καθυστέρηση ανάμεσα στην ενέργεια του χρήστη και την αντίδραση του παιχνιδιού.

3.10 Ζητήματα που αντιμετωπίστηκαν κατά την υλοποίηση

Κατά την ανάπτυξη του "Reddy's Quest" προέκυψαν αρκετά ζητήματα σχεδιασμού και υλοποίησης, τεχνικά και παιδαγωγικά:

- Ασφάλεια εκτέλεσης αυθαίρετου κώδικα: Το σημαντικότερο τεχνικό ζήτημα ήταν η ασφαλής εκτέλεση κώδικα που γράφει ένας άγνωστος, μη έμπιστος χρήστης, χωρίς να είναι εφικτή η χρήση πλήρους απομόνωσης σε επίπεδο λειτουργικού συστήματος (π.χ. container ή ξεχωριστή εικονική μηχανή) εντός μιας απλής desktop εφαρμογής. Η λύση που επιλέχθηκε (συνδυασμός στατικού ελέγχου AST, περιορισμένων builtins και εκτέλεσης με timeout σε ξεχωριστό νήμα, βλ. Ενότητα 3.7) αποτελεί συμβιβασμό ανάμεσα σε ασφάλεια και απλότητα υλοποίησης, κατάλληλο για το πλαίσιο ενός εκπαιδευτικού παιχνιδιού με γνωστό, περιορισμένο εύρος αναμενόμενων λύσεων ανά επίπεδο. Δεν αποτελεί, ωστόσο, πλήρη μηχανισμό απομόνωσης επιπέδου παραγωγής (production-grade), αφού θεωρητικά θα μπορούσαν να υπάρξουν προηγμένες τεχνικές παράκαμψης του στατικού ελέγχου AST που δεν έχουν προβλεφθεί.
- Αποφυγή «πονηρών» λύσεων: Ένα επαναλαμβανόμενο ζήτημα κατά τον σχεδιασμό των κριτηρίων ελέγχου ήταν το πώς θα αποτρέπεται ο χρήστης από το να «κερδίσει» ένα επίπεδο τυπώνοντας απευθείας το επιθυμητό αποτέλεσμα (π.χ. `print("PASS")`) χωρίς να χρησιμοποιήσει την απαιτούμενη δομή. Αυτό οδήγησε στην ανάγκη συνδυασμού ελέγχου δομής (μέσω AST) και ελέγχου αποτελέσματος σε κάθε επίπεδο, αντί για απλή σύγκριση εξόδου, με αντίστοιχη αύξηση της πολυπλοκότητας κάθε συνάρτησης `check_level_X`.
- Ισορροπία δυσκολίας των μηνυμάτων ανατροφοδότησης: Χρειάστηκε προσοχή ώστε τα μηνύματα λάθους να είναι αρκετά συγκεκριμένα για να καθοδηγούν τον μαθητευόμενο (π.χ. «Χρησιμοποίησε `if`, `elif` και `else`»), χωρίς όμως να αποκαλύπτουν απευθείας τη λύση, διατηρώντας έτσι τον χαρακτήρα του puzzle και αποφεύγοντας να μετατραπεί το μήνυμα λάθους σε λύση.
- Υλοποίηση επεξεργαστή κειμένου από την αρχή: Καθώς η Pygame δεν προσφέρει έτοιμο στοιχείο πολυγραμμικού πεδίου κειμένου, χρειάστηκε να αναπτυχθεί από την αρχή ενός απλού επεξεργαστή κώδικα (διαχείριση κέρσορα σε δύο διαστάσεις, εισαγωγή/διαγραφή χαρακτήρων, χειρισμός Backspace/Delete/Enter/Tab, οριζόντια/κάθετη κύλιση), λειτουργία μη τετριμμένη σε ένα περιβάλλον σχεδιασμένο κυρίως για γραφικά και όχι για επεξεργασία κειμένου. Ζητήματα όπως η σωστή διαχείριση της μετακίνησης του κέρσορα ανάμεσα σε γραμμές διαφορετικού μήκους απαίτησαν ιδιαίτερη προσοχή.

- Διαχείριση χρόνου εκτέλεσης χωρίς πάγωμα της διεπαφής: Η ανάγκη εκτέλεσης κώδικα του χρήστη χωρίς να «παγώνει» η κύρια διεπαφή σε περίπτωση ατέρμουνου βρόχου οδήγησε στην υιοθέτηση εκτέλεσης σε ξεχωριστό νήμα με `join(timeout=...)`, μια λύση που απαιτήσε προσεκτικό χειρισμό συγχρονισμού (μέσω `threading.Lock`) ώστε να μην ξεκινούν παράλληλες εκτελέσεις κώδικα από τον ίδιο χρήστη.
- Παρουσίαση κειμένου με αναδίπλωση (text wrapping) στα ελληνικά: Καθώς μεγάλο μέρος του κειμένου (εκφωνήσεις, διάλογοι) είναι στα ελληνικά και εμφανίζεται μέσα σε πάνελ συγκεκριμένου πλάτους, χρειάστηκε η ανάπτυξη λογικής αναδίπλωσης πολλαπλών γραμμών (`draw_wrapped_multiline`) ώστε το κείμενο να προσαρμόζεται σωστά στα διαθέσιμα όρια χωρίς να «κόβεται» ή να ξεχειλίζει από τα πάνελ, λαμβάνοντας υπόψη τα διαφορετικά μήκη των χαρακτήρων της ελληνικής γραμματοσειράς.

3.11 Ενδεικτικά στιγμιότυπα

Στο σημείο αυτό της εργασίας παρατίθενται έξι ενδεικτικά στιγμιότυπα από την εκτέλεση του παιχνιδιού, ώστε ο αναγνώστης να αποκτήσει σαφή εικόνα της διεπαφής χωρίς να χρειάζεται να εγκαταστήσει και να τρέξει την εφαρμογή.

Η Εικόνα 4 δείχνει την αρχική οθόνη (intro) του παιχνιδιού, με τον τίτλο και τον χαρακτήρα Reddy, η οποία αποτελεί την πρώτη εντύπωση και το αφηγηματικό πλαίσιο πριν καν ξεκινήσει η πρώτη αποστολή.

Reddy's Quest: Against Python

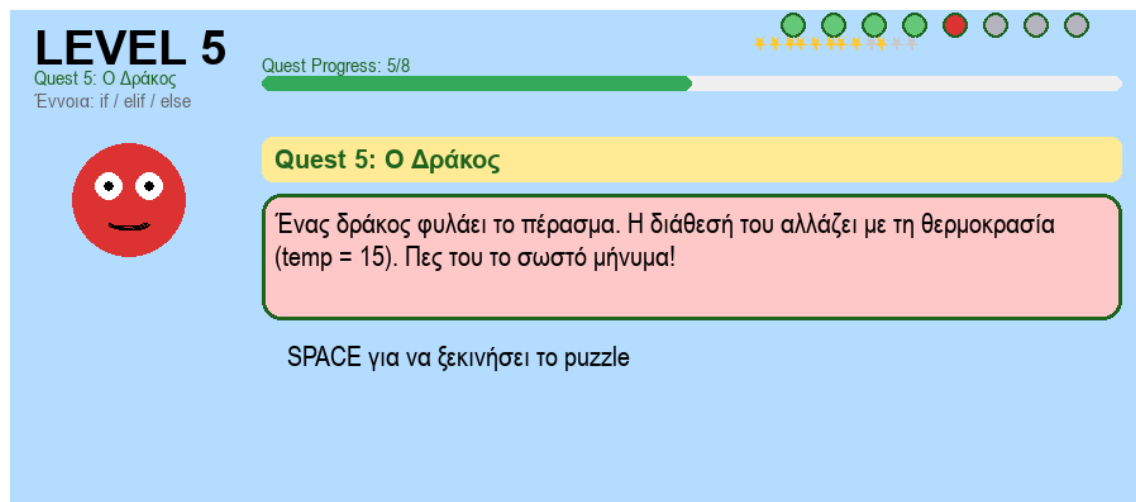


Ο Reddy είναι ένας μικρός προγραμματιστής που θέλει να φτάσει στον Πύργο της Python, όπου βρίσκεται ο Κρύσταλλος της Γνώσης. Για να ανοίξει ο δρόμος, πρέπει να λύσει 8 προγραμματιστικά puzzles. Κάθε puzzle του διδάσκει μια νέα έννοια της Python και τον φέρνει πιο κοντά στον τελικό του στόχο!

Πάτα SPACE για να ξεκινήσεις

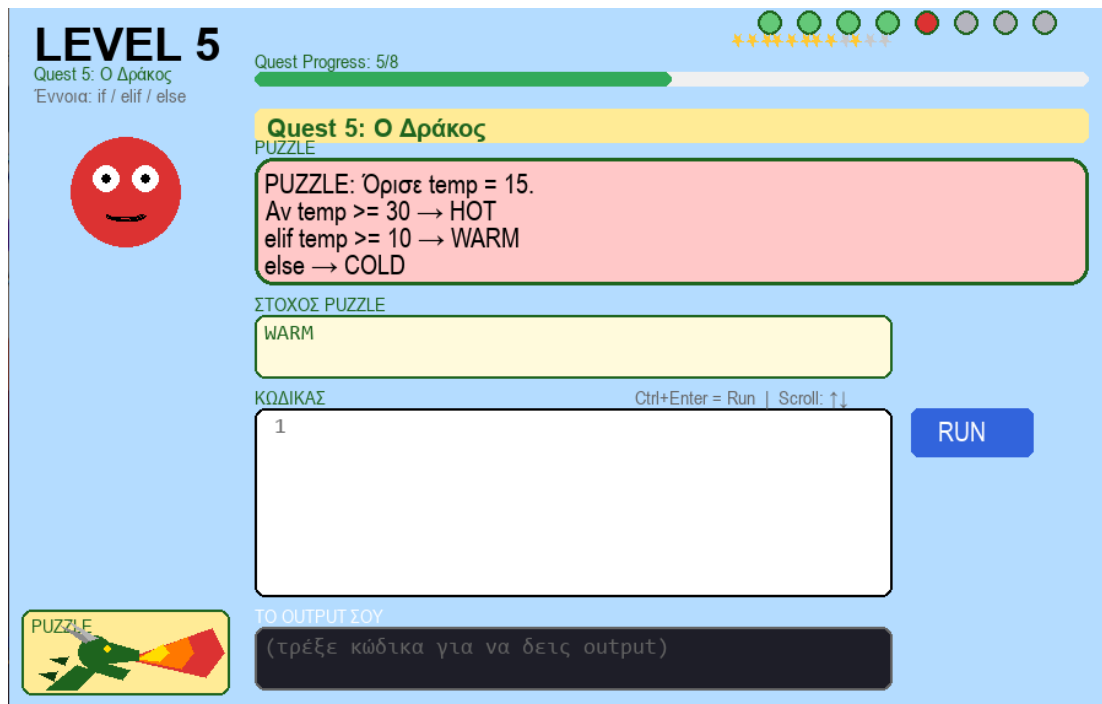
Εικόνα 4: Η αρχική οθόνη (intro) του παιχνιδιού, με τον τίτλο και τον χαρακτήρα Reddy.

Στη συνέχεια, στην Εικόνα 5 απεικονίζεται η φάση διαλόγου (dialogue) ενός επιπέδου, με την αφηγηματική εισαγωγή του quest και το «σύννεφο ομιλίας» του Reddy, μέσω της οποίας πλαισιώνεται αφηγηματικά κάθε puzzle πριν την επίλυσή του.



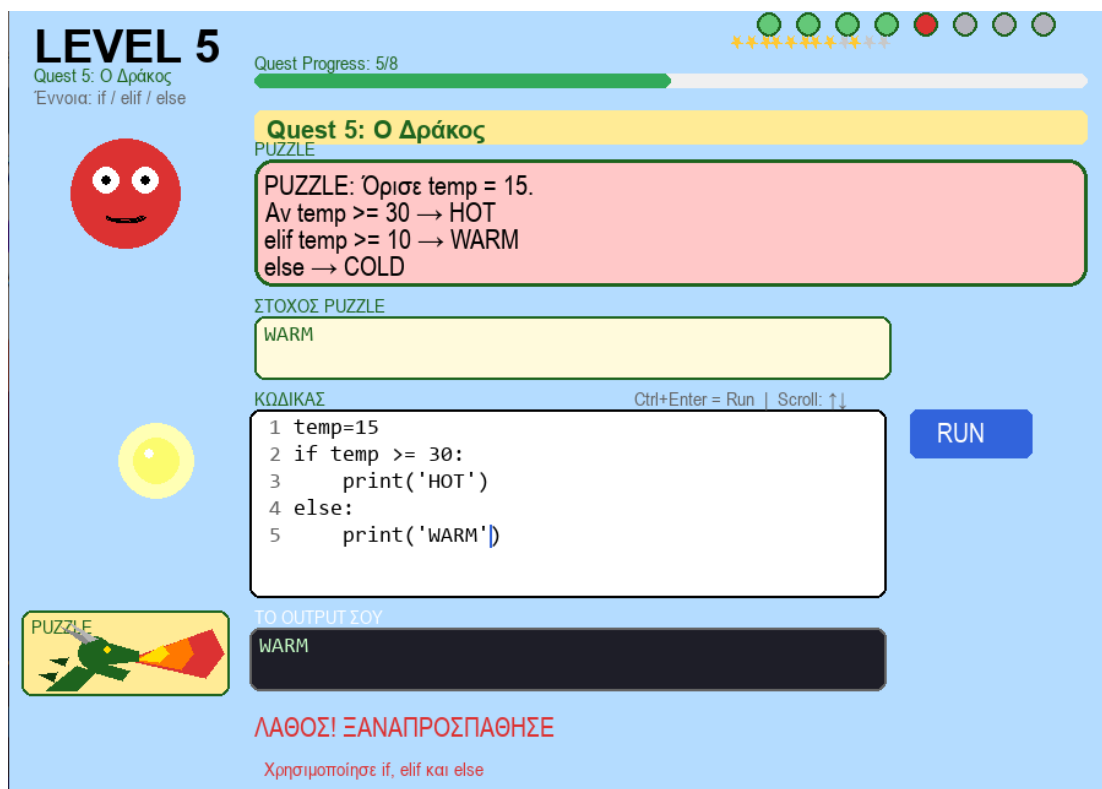
Εικόνα 5: Η φάση διαλόγου (dialogue) στην αρχή ενός επιπέδου, με το σύννεφο ομιλίας του Reddy.

Η σημαντικότερη οθόνη του παιχνιδιού, καθώς αναδεικνύει τον πυρήνα της αλληλεπίδρασης, είναι η φάση επίλυσης (question), η οποία φαίνεται στην Εικόνα 6: διακρίνονται ο editor κώδικα, το panel εξόδου, το κουμπί RUN και το αντίστοιχο οπτικό στοιχείο του επιπέδου (π.χ. ο δράκος στο Quest 5).



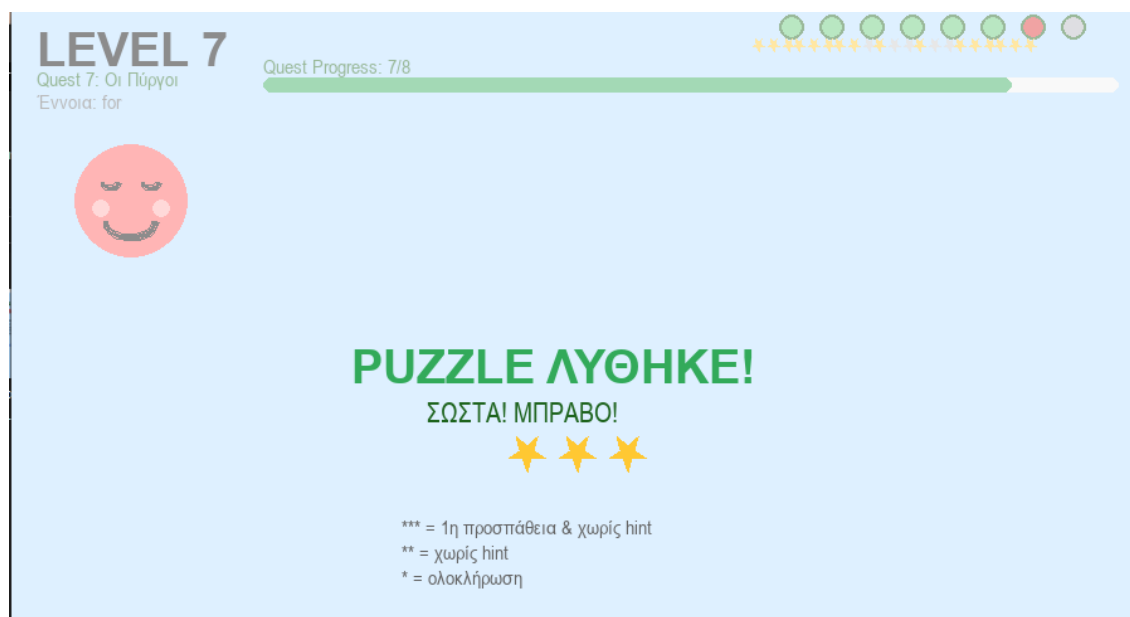
Εικόνα 6: Η φάση επίλυσης (question) ενός επιπέδου, με τον επεξεργαστή κειμένου, το πλαίσιο εξόδου, το κουμπί RUN και το αντίστοιχο οπτικό στοιχείο του επιπέδου.

Η μορφή της αρνητικής ανατροφοδότησης προς τον χρήστη φαίνεται στην Εικόνα 7, όπου εμφανίζει ένα ενεργό μήνυμα λάθους μαζί με το ξεκλειδωμένο κουμπί βοήθηματος.



Εικόνα 7: Ενεργό μήνυμα λάθους και ξεκλειδωμένο βοήθημα κατά την επίλυση επιπέδου.

Αντίστοιχα, η Εικόνα 8 δείχνει τη θετική ανατροφοδότηση που λαμβάνει ο χρήστης όταν εντοπίσει τη σωστή λύση ενός επιπέδου, πριν τη μετάβαση στο επόμενο.



Εικόνα 8: Μήνυμα/ένδειξη επιτυχίας κατά τη σωστή επίλυση ενός επιπέδου.

Τέλος, η Εικόνα 9 δείχνει την οθόνη ολοκλήρωσης (win) του συνόλου του παιχνιδιού, στην οποία συνοψίζεται η συνολική συλλογή αστεριών ανά επίπεδο και αποτυπώνεται η συνολική πρόοδος του παίκτη.



Εικόνα 9: Η οθόνη ολοκλήρωσης (win) με τη συνολική συλλογή αστεριών ανά επίπεδο.

3.12 Απαιτήσεις και εκτέλεση

Το "Reddy's Quest" έχει σχεδιαστεί ώστε να εκτελείται ως απλή desktop εφαρμογή με ελάχιστες απαιτήσεις εγκατάστασης. Για την εκτέλεσή του απαιτείται Python 3.12 και η βιβλιοθήκη Pygame, ενώ η ανάλυση του παραθύρου είναι 900x680 pixel. Η επιλογή αυτών των παραμέτρων έγινε με στόχο την όσο το δυνατόν πιο ομαλή εκτέλεση σε έναν τυπικό σχολικό ή οικιακό υπολογιστή χωρίς πρόσθετη ρύθμιση.

Η εγκατάσταση περιορίζεται ουσιαστικά στην ύπαρξη του κατάλληλου περιβάλλοντος Python και στην εγκατάσταση της Pygame. Το παιχνίδι μπορεί να εκτελεστεί τοπικά, χωρίς ανάγκη σύνδεσης στο διαδίκτυο, και είναι κατάλληλο για χρήση σε εργαστηριακά περιβάλλοντα όπου η πρόσβαση στο διαδίκτυο μπορεί να είναι περιορισμένη.

4 Μελλοντική εργασία

Σε αυτό το κεφάλαιο συζητούνται κατευθύνσεις για τη μελλοντική εξέλιξη του "Reddy's Quest", τόσο ως προς την αξιολόγησή του σε πραγματικές συνθήκες χρήσης όσο και ως προς την επέκταση του περιεχομένου, της τεχνικής υλοποίησης και της διαπαφής του. Όπως σημειώθηκε στο Κεφάλαιο 2 (Ενότητα 2.3), η ΠΜ γενικά πάσχει από έλλειψη αυστηρών εμπειρικών μελετών αποτελεσματικότητας. Η κάλυψη αυτού του κενού ειδικά για το "Reddy's Quest" αποτελεί προτεραιότητα, μαζί με σειρά τεχνικών και παιδαγωγικών επεκτάσεων που θα μπορούσαν να ενισχύσουν περαιτέρω τη χρησιμότητά του.

4.1 Αξιολόγηση σε πραγματικές συνθήκες χρήσης και καταγραφή τηλεμετρίας

Η σημαντικότερη κατεύθυνση για τη συνέχιση αυτής της εργασίας είναι η συστηματική αξιολόγηση του παιχνιδιού με πραγματικούς χρήστες, κάτι που δεν κατέστη εφικτό στο πλαίσιο της παρούσας πτυχιακής. Μια πιλοτική μελέτη θα μπορούσε να διεξαχθεί σε μια ή περισσότερες τάξεις Γυμνασίου ή Λυκείου, σε συνεργασία με εκπαιδευτικό πληροφορικής, με μαθητές να χρησιμοποιούν το "Reddy's Quest" είτε ως μέρος ενός μαθήματος είτε σε ελεγχόμενο εργαστηριακό περιβάλλον.

Για την αξιολόγηση της μαθησιακής αποτελεσματικότητας, θα μπορούσε να ακολουθηθεί σχεδιασμός τεστ όπως: ένα σύντομο τεστ γνώσεων πάνω στις βασικές έννοιες της Python (print, μεταβλητές, συνθήκες, επαναλήψεις) πριν και μετά τη χρήση του παιχνιδιού, ώστε να εκτιμηθεί η βελτίωση της κατανόησης. Παράλληλα, θα ήταν χρήσιμη η σύγκριση με μια ομάδα ελέγχου που διδάσκεται τις ίδιες έννοιες με παραδοσιακό τρόπο (π.χ. διαφάνειες και ασκήσεις σε χαρτί), ώστε να εξεταστεί κατά πόσο η ΠΜ προσφέρει πράγματι μετρήσιμο πλεονέκτημα.

Πέρα από τη μαθησιακή αποτελεσματικότητα, θα είχε ενδιαφέρον η καταγραφή του κινήτρου και της υποκειμενικής εμπειρίας των χρηστών, μέσω σύντομων ερωτηματολογίων μετά τη χρήση (π.χ. ερωτήσεις για το πόσο ευχάριστο, δύσκολο ή κατανοητό βρήκαν

το παιχνίδι, ή για το αν θα ήθελαν να συνεχίσουν να μαθαίνουν προγραμματισμό με παρόμοιο τρόπο), καθώς και άμεση παρατήρηση της χρήσης (π.χ. ποια επίπεδα προκάλεσαν τις περισσότερες λανθασμένες προσπάθειες, πόσο συχνά χρησιμοποιήθηκε το βοήθημα). Τέτοιου είδους ποιοτικά δεδομένα θα βοηθούσαν να επιβεβαιωθεί ή να αναθεωρηθεί η καμπύλη δυσκολίας των οκτώ επιπέδων, καθώς και η ισορροπία ανάμεσα σε παιχνίδι και μάθηση που συζητήθηκε στο Κεφάλαιο 2 (Ενότητα 2.3).

Μια προϋπόθεση για συστηματικότερη, μεγαλύτερης κλίμακας αξιολόγηση τέτοιου τύπου θα ήταν η ενσωμάτωση ενός απλού μηχανισμού καταγραφής δεδομένων χρήσης: αριθμός προσπαθειών ανά επίπεδο, χρόνος που αφιερώθηκε σε κάθε puzzle, συχνότητα χρήσης του βοηθήματος, και τύποι σφαλμάτων που εμφανίστηκαν συχνότερα. Τέτοια δεδομένα, ακόμη και χωρίς σύνδεση στο διαδίκτυο ή αποστολή σε εξωτερικό διακομιστή, θα επέτρεπαν στον εκπαιδευτικό ή τον ίδιο τον δημιουργό του παιχνιδιού να εντοπίσει αντικειμενικά ποια επίπεδα είναι δυσανάλογα δύσκολα ή πού οι μαθητές «κολλάνε» συχνά, χωρίς να χρειάζεται να βασιστεί αποκλειστικά σε υποκειμενική παρατήρηση.

4.2 Επέκταση του περιεχομένου και προσαρμοστική δυσκολία

Τα οκτώ υπάρχοντα επίπεδα καλύπτουν τις πλέον θεμελιώδεις έννοιες της Python (έξοδος, μεταβλητές, αριθμητικές πράξεις, συνθήκες, επαναλήψεις). Μια φυσική συνέχεια θα ήταν η προσθήκη νέων επιπέδων που εισάγουν πιο προχωρημένες, αλλά ακόμη κατάλληλες για αρχάριους, έννοιες: λίστες (`lists`) και βασικές πράξεις πάνω σε αυτές (π.χ. προσπέλαση, προσθήκη στοιχείων), αλφαριθμητικά (`strings`) και βασικές μεθόδους τους, ορισμό και κλήση απλών συναρτήσεων (`def`), καθώς και φωλιασμένες δομές (π.χ. `for` μέσα σε `if` ή `if` μέσα σε `for` σε μεγαλύτερο βάθος από ό,τι στο Quest 8). Χάρη στον διαχωρισμό περιεχομένου που περιγράφηκε στην Ενότητα 3.6 (η δομή δεδομένων `levels`), η προσθήκη νέων επιπέδων δεν απαιτεί αλλαγές στην υπόλοιπη αρχιτεκτονική του παιχνιδιού, παρά μόνο νέες εγγραφές επιπέδων και αντίστοιχες συναρτήσεις `check_level_X` και βοηθητικές συναρτήσεις ανάλυσης AST όπου χρειάζεται.

Πέρα από νέες έννοιες, θα μπορούσαν να προστεθούν εναλλακτικοί τρόποι επίλυσης για τα υπάρχοντα επίπεδα: αντί για μία μοναδική «σωστή» δομή ανά επίπεδο, ορισμένα puzzle θα μπορούσαν να δέχονται περισσότερες από μία έγκυρες προσεγγίσεις (π.χ. στο Quest 6, να γίνεται αποδεκτή τόσο η λύση με `while` όσο και μια εναλλακτική με `for`,

αν το επίπεδο επαναπροσδιοριστεί ως άσκηση επανάληψης γενικότερα), κάτι που θα απαιτούσε πιο ευέλικτες συναρτήσεις ελέγχου, αλλά θα πλησίαζε περισσότερο την ελευθερία που έχει ένας προγραμματιστής στην πράξη.

Πέρα όμως από το τι περιεχόμενο διδάσκεται, ζήτημα αποτελεί και το πώς προσαρμόζεται η δυσκολία στον κάθε χρήστη. Όπως συζητήθηκε στο Κεφάλαιο 2 (Ενότητα 2.3), η ανομοιογένεια του υποβάθρου των μαθητών αποτελεί σημαντική πρόκληση για μια σταθερή καμπύλη δυσκολίας. Μια πιθανή εξέλιξη θα ήταν η εισαγωγή στοιχείων προσαρμοστικής δυσκολίας: για παράδειγμα, ο αριθμός των λανθασμένων προσπαθειών θα μπορούσαν να χρησιμοποιηθούν όχι μόνο για τον υπολογισμό αστεριών (όπως ήδη γίνεται, βλ. Ενότητα 3.9), αλλά και για δυναμική προσαρμογή π.χ. προσφορά ενός επιπλέον, απλούστερου ενδιάμεσου puzzle σε έναν χρήστη που δυσκολεύεται επίμονα σε μια συγκεκριμένη έννοια, ή αντίστροφα, προαιρετικών «προκλήσεων μπόνους» για χρήστες που ολοκληρώνουν τα επίπεδα χωρίς βοήθημα, ώστε να αποφεύγεται η μονοτονία,.

4.3 Βελτιώσεις διεπαφής, προσβασιμότητας και εργαλεία υποστήριξης εκπαιδευτικού

Η τρέχουσα διεπαφή, αν και λειτουργική, θα μπορούσε να βελτιωθεί ως προς την προσβασιμότητα: προσθήκη ρύθμισης μεγέθους γραμματοσειράς για χρήστες με προβλήματα όρασης, χρωματική παλέτα φιλική για χρήστες με αχρωματοψία, δυνατότητα απενεργοποίησης των ηχητικών εφέ ή των πιο έντονων οπτικών εφέ (όπως η δόνηση της οθόνης) για χρήστες που τα βρίσκουν ενοχλητικά ή αποσπαστικά, καθώς και πληρέστερη υποστήριξη πλοήγησης αποκλειστικά μέσω πληκτρολογίου. Επιπλέον, θα ήταν χρήσιμη η προσθήκη ρύθμισης ανάλυσης οθόνης ή αυτόματης προσαρμογής στο μέγεθος του παραθύρου, αφού η τρέχουσα υλοποίηση χρησιμοποιεί σταθερή ανάλυση 900x680 pixel (βλ. Ενότητα 3.12), κάτι που ενδέχεται να μη ταιριάζει εξίσου καλά σε όλες τις οθόνες εργαστηριακών υπολογιστών.

Πέρα από τον ίδιο τον μαθητή, ένα δεύτερο κοινό που θα ωφελούνταν από βελτιώσεις στη διεπαφή είναι ο εκπαιδευτικός. Για να ενισχυθεί ο ρόλος του "Reddy's Quest" ως συμπληρωματικό εργαλείο μέσα στην τάξη (βλ. Ενότητα 3.1), θα ήταν χρήσιμη η ανάπτυξη απλών εργαλείων υποστήριξης για τον εκπαιδευτικό. Για παράδειγμα, μια συνοπτική αναφορά προόδου ανά μαθητή ή ανά τμήμα (πόσα επίπεδα έχουν ολοκληρωθεί, πόσα αστέρια συνολικά, ποια επίπεδα προκάλεσαν τις περισσότερες δυσκολίες), βασι-

σμένη στα δεδομένα τηλεμετρίας της Ενότητας 4.1. Κάτι τέτοιο θα επέτρεπε στον εκπαιδευτικό να εντοπίζει μαθητές που ίσως χρειάζονται επιπλέον υποστήριξη, χωρίς να χρειάζεται να παρακολουθεί ο ίδιος σε πραγματικό χρόνο κάθε μαθητή ξεχωριστά.

4.4 Ασφάλεια, μηχανισμός αξιολόγησης και τεχνική επέκταση

Όπως αναγνωρίστηκε στην Ενότητα 3.10, ο μηχανισμός απομόνωσης (sandboxing) του παιχνιδιού (στατικός έλεγχος AST, περιορισμένα builtins, εκτέλεση με timeout) αποτελεί κατάλληλο συμβιβασμό για το πλαίσιο ενός εκπαιδευτικού παιχνιδιού, αλλά δεν είναι πλήρης μηχανισμός απομόνωσης επιπέδου παραγωγής. Μελλοντική εργασία θα μπορούσε να επικεντρωθεί στην ενίσχυση αυτού του μηχανισμού, για παράδειγμα μέσω πιο συστηματικού ελέγχου που θα δοκιμάζουν σκόπιμα κακόβουλες εισόδους κώδικα, ώστε να εντοπιστούν πιθανοί τρόποι παράκαμψης του στατικού ελέγχου AST πριν αυτοί εντοπιστούν από πραγματικούς χρήστες. Αντίστοιχα, ο μηχανισμός αξιολόγησης λύσεων (Ενότητα 3.8) θα μπορούσε να εμπλουτιστεί με περισσότερες, πιο λεπτομερείς κατηγορίες μηνυμάτων λάθους, βασισμένες σε πραγματικά δεδομένα από την αξιολόγηση της Ενότητας 4.1 (π.χ. αν φανεί ότι πολλοί μαθητές κάνουν ένα συγκεκριμένο, προβλέψιμο λάθος σε κάποιο επίπεδο, θα μπορούσε να προστεθεί ειδικό μήνυμα γι' αυτή την περίπτωση).

Σε τεχνικό επίπεδο γενικότερα, η τρέχουσα υλοποίηση ως ενιαίο αρχείο Python με Pygame (βλ. Ενότητα 3.6) θα μπορούσε να εξελιχθεί προς διάφορες κατευθύνσεις, ανάλογα με τον στόχο. Μια πρώτη κατεύθυνση θα ήταν η εσωτερική αναδιάρθρωση του κώδικα σε πολλαπλά αρχεία (π.χ. ξεχωριστό module για sandbox και validation, ένα για UI, ένα για levels), κάτι που θα διευκόλυνε τη συντήρηση και τη συνεργατική ανάπτυξη σε μεγαλύτερη κλίμακα, με το αντίστοιχο κόστος αυξημένης πολυπλοκότητας για ένα έργο αυτού του μεγέθους. Μια δεύτερη, πιο φιλόδοξη κατεύθυνση θα ήταν η μεταφορά της εφαρμογής σε περιβάλλον προσβάσιμο μέσω πρόγραμμα περιήγησης (π.χ. αξιοποιώντας εργαλεία μεταγλώττισης Python σε WebAssembly). Τέλος, θα μπορούσε να εξεταστεί η προσθήκη δυνατότητας αποθήκευσης προόδου ανάμεσα σε εκτελέσεις, ώστε ο χρήστης να μη χρειάζεται να ολοκληρώνει όλα τα επίπεδα σε μία συνεδρία, κάτι που απουσιάζει από την τρέχουσα έκδοση.

Συνολικά, οι παραπάνω κατευθύνσεις δεν είναι ανεξάρτητες μεταξύ τους: η συστηματική αξιολόγηση (Ενότητα 4.1) αποτελεί προϋπόθεση ώστε οι υπόλοιπες επεκτάσεις

(περιεχόμενο, δυσκολία, διεπαφή, εργαλεία εκπαιδευτικού) να βασιστούν σε πραγματικά δεδομένα χρήσης και όχι μόνο σε θεωρητικές παραδοχές, κλείνοντας έτσι τον κύκλο ανάμεσα στον αρχικό σχεδιασμό του παιχνιδιού και την πραγματική του αποτελεσματικότητα ως εκπαιδευτικό εργαλείο.

Βιβλιογραφία

- [1] Csikszentmihalyi, M., *Flow: The Psychology of Optimal Experience*, Harper & Row, New York, NY, USA, 1990.
- [2] Deterding, S., Dixon, D., Khaled, R., & Nacke, L., "From game design elements to gamefulness: Defining 'gamification'", in *Proceedings of the 15th International Academic MindTrek Conference: Envisioning Future Media Environments*, ACM, pp. 9–15, Tampere, Finland, September 28–30, 2011.
- [3] Garris, R., Ahlers, R., & Driskell, J. E., "Games, Motivation, and Learning: A Research and Practice Model", *Simulation & Gaming*, vol. 33, no. 4, pp. 441–467, 2002.
- [4] Gee, J. P., *What Video Games Have to Teach Us About Learning and Literacy*, Palgrave Macmillan, 2003.
- [5] Gros, B., "Digital games in education: The design of games-based learning environments", *Journal of Research on Technology in Education*, vol. 40, no. 1, pp. 23–38, 2007.
- [6] Hamari, J., Koivisto, J., & Sarsa, H., "Does Gamification Work? — A Literature Review of Empirical Studies on Gamification", in *2014 47th Hawaii International Conference on System Sciences*, IEEE, pp. 3025–3034, Waikoloa, HI, USA, January 6–9, 2014.
- [7] Kiili, K., "Digital Game-Based Learning: Towards an Experiential Gaming Model", *The Internet and Higher Education*, vol. 8, no. 1, pp. 13–24, 2005.
- [8] Lister, R., Simon, B., Thompson, E., Whalley, J. L., Prasad, C., & Adams, E., "A multi-national study of reading and tracing skills in novice programmers", in *Proceedings of the 9th Annual SIGCSE Conference on Innovation and Technology in Computer Science Education (ITiCSE 2004)*, ACM Press, pp. 196–200, Leeds, United Kingdom, June 28–30, 2004.
- [9] Malone, T. W., "Toward a theory of intrinsically motivating instruction", *Cognitive Science*, vol. 5, no. 4, pp. 333–369, 1981

- [10] Michael, D. R., & Chen, S. L., *Serious Games: Games That Educate, Train, and Inform*, Thomson Course Technology, 2006.
- [11] Papert, S., *Mindstorms: Children, Computers, and Powerful Ideas*, Basic Books, 1st Edition, New York, NY, USA, ISBN 978-0-465-04657-9, 232 pp., 1980.
- [12] Prensky, M., *Digital Game-Based Learning*, McGraw-Hill, 2001.
- [13] Pygame Community, "Pygame documentation" — Επίσημη τεκμηρίωση της βιβλιοθήκης Pygame. <https://www.pygame.org/docs/> [Ημερομηνία πρόσβασης: 13/07/2026]
- [14] Python Software Foundation, "Python 3.12 documentation" — Επίσημη τεκμηρίωση της γλώσσας Python. <https://docs.python.org/3.12/> — Πρόσβαση: 13/07/2026
- [15] Python Software Foundation, "ast — Abstract Syntax Trees". <https://docs.python.org/3.12/library/ast.html> [13/07/2026].
- [16] Resnick, M., Maloney, J., Monroy-Hernández, A., Rusk, N., Eastmond, E., Brennan, K., Millner, A., Rosenbaum, E., Silver, J., Silverman, B., & Kafai, Y., "Scratch: Programming for all", *Communications of the ACM*, vol. 52, no. 11, pp. 60–67, 2009.
- [17] Shernoff, D. J., Csikszentmihalyi, M., Schneider, B., & Shernoff, E. S., "Student engagement in high school classrooms from the perspective of flow theory", *School Psychology Quarterly*, vol. 18, no. 2, pp. 158–176, 2003.
- [18] Wiggins, G., & McTighe, J., *Understanding by Design*, Expanded 2nd Edition, Association for Supervision and Curriculum Development (ASCD), Alexandria, VA, USA, ISBN 978-1-4166-0035-0, 370 pp., 2005.

Παράρτημα Α

A.1 Κώδικας πλήρους λίστας levels

Στο σημείο αυτό παρατίθεται η πλήρης λίστα levels που περιγράφεται στην Ενότητα 3.6, με τις εγγραφές και των 8 επιπέδων του παιχνιδιού.

```
levels = [
    {
        "concept": "print",
        "quest_title": "Quest 1: Η Πύλη",
        "dialogue": "Ο πρώτος φρουρός δεν αναγνωρίζει τον Reddy. Η πύλη θα ανοίξει μόνο αν εμφανιστεί το σωστό μήνυμα.",
        "question": "PUZZLE: Στείλε το μυστικό μήνυμα στην πόρτα.\nΠρέπει να εμφανιστεί ακριβώς:\nHello, Reddy!",
        "expected": "Hello, Reddy!",
        "hint": "print('Hello, Reddy!')",
        "check": check_level_1,
        "visual": "gate",
    },
    {
        "concept": "μεταβλητές",
        "quest_title": "Quest 2: Ο Κρύσταλλος",
        "dialogue": "Μέσα στον πύργο υπάρχει ένας μαγικός κρύσταλλος που αποθηκεύει πληροφορίες. Αποθήκευσε το όνομα της γλώσσας!",
        "question": "PUZZLE: Αποθήκευσε name = \"Python\" και τύπωσε τη μεταβλητή name.",
        "expected": "Python",
        "hint": "name = \"Python\"\nprint(name)",
        "check": check_level_2,
        "visual": "crystal",
    },
    {
        "concept": "πράξεις",
        "quest_title": "Quest 3: Η Κλειδαριά",
        "dialogue": "Η τεράστια μεταλλική κλειδαριά ανοίγει μόνο αν υπολογιστεί σωστά ο μυστικός αριθμός. Υπολόγισε το 12 + 8.",
        "question": "PUZZLE: Υπολόγισε με πράξη (+, -, *, /) και τύπωσε το αποτέλεσμα του 12 + 8.",
        "expected": "20",
        "hint": "print(12 + 8)",
        "check": check_level_3,
        "visual": "lock",
    },
    {
        "concept": "if / else",
        "quest_title": "Quest 4: Η Εύλινη Πύλη",
        "dialogue": "Μια μεγάλη ξύλινη πόρτα αποφασίζει ποιος περνάει. Αν score >= 5 → PASS, αλλιώς FAIL. Το score είναι 7!",
        "question": "PUZZLE: Όρισε score = 7.\nΑν score >= 5 τύπωσε PASS, αλλιώς FAIL.",
        "expected": "PASS",
        "hint": "score = 7\nif score >= 5:\n    print('PASS')\nelse:\n    print('FAIL')",
    },
]
```

```

        "check": check_level_4,
        "visual": "door",
    },
    {
        "concept": "if / elif / else",
        "quest_title": "Quest 5: Ο Δράκος",
        "dialogue": "Ένας δράκος φυλάει το πέρασμα. Η διάθεσή του αλλάζει με τη θερμο-
κρασία (temp = 15). Πες του το σωστό μήνυμα!",
        "question": "PUZZLE: Όρισε temp = 15.\nΑν temp >= 30 → HOT\nelif temp >= 10 →
WARM\nelse → COLD",
        "expected": "WARM",
        "hint": "temp = 15\nif temp >= 30:\n    print('HOT')\nelif temp >= 10:\n
print('WARM')\nelse:\n    print('COLD')",
        "check": check_level_5,
        "visual": "dragon",
    },
    {
        "concept": "while",
        "quest_title": "Quest 6: Ο Μηχανισμός",
        "dialogue": "Ένας αρχαίος μηχανισμός χρειάζεται να μετρήσει μέχρι το 3 για να ε-
νεργοποιηθεί. Χρησιμοποίησε while!",
        "question": "PUZZLE: Με while loop, τύπωσε τους αριθμούς 1, 2, 3\n(μία γραμμή η
καθεμία).",
        "expected": "1\n2\n3",
        "hint": "i = 1\nwhile i <= 3:\n    print(i)\n    i += 1",
        "check": check_level_6,
        "visual": "gear",
    },
    {
        "concept": "for",
        "quest_title": "Quest 7: Οι Πύργοι",
        "dialogue": "Τέσσερις πύργοι επικοινωνίας πρέπει να ενεργοποιηθούν ένας-ένας.
Κάθε '!' ανάβει έναν πύργο!",
        "question": "PUZZLE: Με for και range, τύπωσε '\n' 4 φορές\n(μία σε κάθε
γραμμή).",
        "expected": "!\n!\n!\n!",
        "hint": "for i in range(4):\n    print('!')",
        "check": check_level_7,
        "visual": "towers",
    },
    {
        "concept": "συνδυασμός",
        "quest_title": "Quest 8: Η Γέφυρα",
        "dialogue": "Ο Πύργος της Python είναι απέναντί! Η γέφυρα έχει καταστραφεί.
Χτίσε τα πλακίδια: Άρτια = Reddy, μονά = -.",
        "question": "PUZZLE: Για κάθε αριθμό 1 έως 5:\nαν είναι άρτιος → Reddy\nαλλιώς →
-\n(Χτίσε τη γέφυρα με print!)",
        "expected": "-\nReddy\n-\nReddy\n-",
        "hint": "for n in range(1, 6):\n    if n % 2 == 0:\n        print('Reddy')\n
else:\n    print('-')",
        "check": check_level_8,
        "visual": "bridge",
    },
}
]

```

A.2 Μηχανισμός ασφαλούς εκτέλεσης κώδικας (Sandbox)

Στο σημείο αυτό παρατίθεται ο πλήρης κώδικας του μηχανισμού sandboxing που περιγράφεται στην Ενότητα 3.7: οι σταθερές ασφαλείας, ο στατικός έλεγχος AST, και η εκτέλεση με χρονικό όριο σε ξεχωριστό νήμα.

```
SAFE_BUILTINS = {
    "print": print, "range": range, "len": len,
    "int": int, "float": float, "str": str, "bool": bool,
}

BANNED_NAMES = {
    "open", "eval", "exec", "compile", "__import__",
    "globals", "locals", "vars", "input", "exit", "quit",
}

EXECUTION_TIMEOUT = 2.0
_execution_thread = None
_execution_lock = threading.Lock()

def validate_ast(tree):
    # Ελέγχει αν το Abstract Syntax Tree (AST) περιέχει απαγορευμένες εντολές.
    # Επιστρέφει: tuple: (bool, str) - (True, '') αν ο κώδικας είναι ασφαλής, (False,
    # error_msg) αν βρεθεί απαγορευμένη εντολή
    # Ελέγχει για:
    # - Import statements (import, from ... import)
    # - Απαγορευμένα ονόματα (eval, exec, open, κλπ)
    # - Πρόσβαση σε attributes (security risk)
    # - while True loops (infinite loop prevention)
    # - break/continue statements
    for node in ast.walk(tree):
        if isinstance(node, (ast.Import, ast.ImportFrom)):
            return False, " Δεν επιτρέπεται η χρήση import."
        if isinstance(node, ast.Name) and node.id in BANNED_NAMES:
            return False, f" Η εντολή '{node.id}' δεν επιτρέπεται."
        if isinstance(node, ast.Attribute):
            return False, " Δεν επιτρέπεται πρόσβαση σε εσωτερικά αντικείμενα."
        if isinstance(node, ast.While):
            if isinstance(node.test, ast.Constant) and node.test.value is True:
                return False, " Δεν επιτρέπεται while True."
        if isinstance(node, (ast.Break, ast.Continue)):
            return False, " Δεν επιτρέπεται break/continue."
        if isinstance(node, ast.Call):
            if isinstance(node.func, ast.Name) and node.func.id in BANNED_NAMES:
                return False, f" Η εντολή '{node.func.id}' δεν επιτρέπεται."
    return True, ""

def _execute_in_thread(code, result_container):
    # Εκτελεί κώδικα Python σε ξεχωριστό thread για υποστήριξη timeout.
    # Χρησιμοποιείται σε συνδυασμό με threading.Thread.join(timeout) για την αποφυγή
    # infinite loops στον κώδικα του χρήστη.
    # Ο κώδικας εκτελείται με περιορισμένα builtins (SAFE_BUILTINS) για λόγους ασφαλείας.
    try:
        output = io.StringIO()
        with contextlib.redirect_stdout(output):
```

```

        exec(code, {"__builtins__": SAFE_BUILTINS})
        result_container["output"] = output.getvalue()
        result_container["error"] = None
    except Exception as e:
        result_container["output"] = ""
        result_container["error"] = e

def run_user_code(code, check_func):
# Εκτελεί τον κώδικα του χρήστη σε ασφαλές sandbox και επιστρέφει το αποτέλεσμα.
# Επιστρέφει:
#     tuple: (success: bool, output: str, error: str ή None)
#             success: True αν ο κώδικας πέρασε τον έλεγχο
#             output: Το stdout από την εκτέλεση
#             error: Μήνυμα σφάλματος (αν υπάρχει)
# Διαδικασία:
# 1. Έλεγχος κενού κώδικα και μέγεθους
# 2. Parsing σε AST και έλεγχος syntax
# 3. Validation AST για απαγορευμένες εντολές
# 4. Εκτέλεση σε thread με timeout (EXECUTION_TIMEOUT)
# 5. Χειρισμός exceptions και επιστροφή κατάλληλου μηνύματος
# 6. Έλεγχος αποτελέσματος μέσω check_func
global _execution_thread
if not code.strip():
    return False, "", "Γράψε κώδικα πριν πατήσεις RUN"
if len(code) > 1500:
    return False, "", "Ο κώδικας είναι πολύ μεγάλος."
if len(code.splitlines()) > 80:
    return False, "", "Πάρα πολλές γραμμές κώδικα."
try:
    tree = ast.parse(code)
except SyntaxError as e:
    return False, "", f"SyntaxError: {e.msg} (γραμμή {e.lineno})"
ok_ast, ast_msg = validate_ast(tree)
if not ok_ast:
    return False, "", ast_msg
with _execution_lock:
    if _execution_thread is not None and _execution_thread.is_alive():
        return False, "", "Περίμενε... ο προηγούμενος κώδικας ακόμα τρέχει."
    result_container = {"output": "", "error": None}
    _execution_thread = threading.Thread(
        target=_execute_in_thread, args=(code, result_container), daemon=True,
    )
    _execution_thread.start()
    _execution_thread.join(timeout=EXECUTION_TIMEOUT)
if _execution_thread.is_alive():
    return False, "", (
        f"Υπερβήκατε το όριο χρόνου ({EXECUTION_TIMEOUT:.0f} δευτ.). "
        "Πιθανός ατέρμονος βρόχος – έλεγχε τη συνθήκη του while/for."
    )
if result_container["error"] is not None:
    e = result_container["error"]
    if isinstance(e, NameError):
        return False, "", f"Μεταβλητή που δεν υπάρχει.\n{e}"
    if isinstance(e, TypeError):
        return False, "", f"Λάθος τύπος δεδομένων.\n{e}"
    if isinstance(e, ZeroDivisionError):
        return False, "", "Δεν μπορείς να διαιρέσεις με το μηδέν."
    if isinstance(e, IndentationError):
        return False, "", "Λάθος indentation."
    return False, "", f"Σφάλμα: {e}"

```

```
result = result_container["output"]
ok, feedback = check_func(code, tree, result)
if ok:
    return True, result, None
return False, result, feedback
```